

Denny Chrisnanwwda 2502124913

Seminar II

**OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON
BERBASIS PROTOKOL FINS PADA PT XYZ**



Oleh

**COMPUTER SCIENCE STUDY PROGRAM
BINUS ONLINE LEARNING
UNIVERSITAS BINA NUSANTARA
JAKARTA**

2025

DAFTAR ISI

DAFTAR ISI	i
DAFTAR TABEL	ii
DAFTAR GAMBAR	iii
DAFTAR LAMPIRAN	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Identifikasi Masalah	2
1.3 Rumusan Masalah	3
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	3
1.6 Ruang Lingkup Penelitian	3
1.7 Hipotesis Penelitian	4
1.8 Sistematika Penulisan	4
BAB II TINJAUAN PUSTAKA	5
2.1 Tinjauan Studi	5
2.2 Tinjauan Pustaka	7
2.2.1 Objek Penelitian	7
2.2.2 Perusahaan Manufaktur dan Kebutuhan Otomasi	7
2.2.3 Programmable Logic Controller (PLC)	7
2.2.4 Protokol Komunikasi Industri dan FINS	8
2.2.5 HMI dan Visualisasi Data	8
2.2.6 Teknologi Web Modern: HTML, CSS, JavaScript, TypeScript, Node.js, Angular	8
2.2.7 HMI Open-Source dan Vendor Lock-In	10
2.2.8 Electron: Aplikasi Desktop Berbasis Web	10
2.2.9 Pengujian dan Validasi Sistem HMI	11

2.3	Kerangka Pemikiran	12
BAB III METODOLOGI PENELITIAN		14
3.1	Tahapan Penelitian	14
3.2	Alat dan Bahan	16
3.2.1	Perangkat Keras	16
3.2.2	Perangkat Lunak	16
3.2.3	Library Tambahan	16
3.2.4	Platform dan Arsitektur Aplikasi	16
3.2.5	Topologi Sistem dan Komunikasi FINS	17
3.3	Modifikasi UI (Frontend Angular)	18
3.4	Modifikasi Backend (Node.js Konektor FINS)	20
3.5	Metode Pengujian	24
3.5.1	Black-Box Testing	24
3.5.2	White-Box Testing	25
3.5.3	Metrik Evaluasi	25
3.5.4	Validasi dan Reproducibility	26
3.6	Evaluasi Berbasis Responden	26
BAB IV HASIL DAN DISKUSI		28
4.1	Hasil Uji Coba Pustaka omron-fins	28
4.1.1	Konfigurasi Lingkungan Uji	28
4.1.2	Instalasi Pustaka omron-fins	28
4.1.3	Metode Pengujian	29
4.1.4	Hasil dan Analisis Pengujian	30
4.2	Konfigurasi Sistem	32
4.2.1	Topologi dan Arsitektur Sistem	32
4.2.2	Konfigurasi Perangkat Keras	34
4.2.3	Konfigurasi Perangkat Lunak	34
4.2.4	Verifikasi Koneksi Awal	35
4.3	Hasil Implementasi Sistem	35
4.3.1	Fungsionalitas yang Tercapai	35

4.3.2	Hasil Modifikasi Antarmuka	36
4.3.3	Hasil Modifikasi Backend	37
4.3.4	Demonstrasi Running dan Komunikasi Realtime dengan PLC .	39
4.3.5	Demo Alarm Waktu Nyata pada Sistem HMI	40
4.4	Hasil Evaluasi Sistem	41
4.4.1	Hasil White-Box Testing	42
4.4.2	Hasil Black-Box Testing	43
4.4.3	Pengujian Teknis	43
4.4.4	Observasi Fault Tolerance	50
4.4.5	Evaluasi Berbasis Responden	50
4.4.6	Pembahasan Singkat	52
BAB V	KESIMPULAN DAN SARAN	53
5.1	Kesimpulan	53
5.2	Saran	54
DAFTAR PUSTAKA		55
LAMPIRAN		58
BAB A	Kode Test Pustaka Omron-Fins	59
BAB B	Kode Modifikasi Backend	62
BAB C	Tabulasi Data Penelitian	70

DAFTAR TABEL

2.1	Tabel Tinjauan Studi	5
3.1	Black-Box Testing Konektor FINS	24
3.2	White-Box Testing Konektor FINS	25
3.3	Metrik dan Target KPI Evaluasi Sistem HMI dengan Konektor FINS .	26
4.1	Konfigurasi lingkungan uji coba pustaka <i>omron-fins</i>	28
4.2	Konfigurasi IP perangkat dalam sistem	34
4.3	Parameter konfigurasi koneksi FINS di sistem HMI	35
4.4	Hasil White-Box Testing Konektor FINS	42
4.5	Hasil Black-Box Testing Konektor FINS	43
4.6	Ringkasan hasil pengujian teknis	44
4.7	Data hasil evaluasi responden (R1–R10) beserta rata-rata	51
4.8	Ringkasan hasil evaluasi berbasis responden	51

DAFTAR GAMBAR

2.1	Diagram kerangka pemikiran penelitian	12
3.1	Flowchart Tahapan Penelitian	14
3.2	Topologi Sistem SCADA FUXA dengan Protokol FINS (PC, PLC, Hub, Tablet)	17
3.3	Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi perangkat FINS	18
3.4	Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi tag perangkat FINS	19
4.1	Instalasi pustaka <code>omron-fins</code> melalui perintah <code>npm install</code> <code>node-red-contrib-omron-fins</code>	29
4.2	Skrip pengujian pustaka <code>omron-fins</code>	30
4.3	Tangkapan layar hasil uji coba pustaka <code>omron-fins</code>	31
4.4	Cuplikan hasil analisis paket FINS pada Wireshark	31
4.5	Topologi sistem HMI FUXA dengan konektor FINS	33
4.6	Hasil UI — Setting Device: form konfigurasi device FINS (IP, port, SA1, DA1, protocol).	36
4.7	Hasil UI — Setting Tag: dialog edit tag untuk konfigurasi memaddress, address, tipe data.	36
4.8	Potongan kode modul konektor FINS pada sisi backend	38
4.9	Demo sistem HMI yang terkoneksi dengan PLC.	39
4.10	Tampilan demo alarm pada HMI ketika suhu oli melebihi batas aman.	40
4.11	Tampilan log hasil pengujian latensi baca/tulis FINS antara FUXA dan PLC Omron	45
4.12	Tampilan log hasil pengujian persentase sukses komunikas	47
4.13	Pemantauan penggunaan CPU dan memori oleh frontend (Edge) dan backend (Node.js)	48
4.14	Batasan interval waktu deteksi alarm sistem HMI	49
4.15	Tangkapan layar proses inisialisasi aplikasi	49

DAFTAR LAMPIRAN

Kode Test Pustaka Omron-Fins	59
Kode Modifikasi Backend	62
Tabulasi Data Penelitian	70

BAB I

PENDAHULUAN

1.1 Latar Belakang

Programmable logic controller (PLC) merupakan perangkat utama dalam otomasi industri. Fungsi utama dari PLC adalah memproses dan mengendalikan mesin dengan pengetahuan yang terbatas mengenai pemrograman komputer (koondhar2023role). Di lingkungan PT XYZ, perusahaan manufaktur komponen otomotif multinasional, sebagian besar mesin produksi telah menggunakan PLC merek Omron. Data internal menunjukkan bahwa lebih dari 90% mesin produksi pada perusahaan mengandalkan PLC Omron sebagai pusat kendali. Tingginya tingkat adopsi PLC Omron pada lini produksi menjadikannya objek penelitian yang relevan dalam konteks integrasi sistem kendali dengan teknologi digital modern.

Untuk mendukung interaksi antara operator dan mesin yang dikendalikan oleh PLC, dibutuhkan *human-machine interface* (HMI). HMI berperan menampilkan data proses, memberikan kontrol interaktif, serta mendukung pengawasan visual yang memudahkan manajemen produksi (mujawar2023'hmi'intro). Selama ini, banyak industri mengandalkan sistem HMI komersial. Sistem HMI komersial memiliki keunggulan dalam pengendalian, pengawasan, dan analisis untuk mengurangi *downtime*, biaya, serta menjaga standar kualitas (Sawal, 2025). Namun, ketergantungan pada vendor tertentu sering menimbulkan masalah biaya lisensi, keterbatasan fleksibilitas, dan risiko *vendor lock-in* (Gularte, 2025). Sebagai alternatif, pendekatan sumber terbuka (*open source*) menawarkan efisiensi biaya, fleksibilitas, serta kemudahan kustomisasi. Salah satu platform HMI/SCADA *open source* yang berkembang adalah FUXA, yang berbasis web, ringan, serta mendukung akses jarak jauh dan integrasi IoT (Chen, 2025). Karakteristik ini menjadikan FUXA sebagai sistem HMI sumber terbuka yang potensial untuk mendukung kebutuhan otomasi cerdas di industri manufaktur.

Pada PLC Omron, komunikasi data umumnya dilakukan menggunakan protokol *factory interface network service* (FINS), yakni standar komunikasi yang memungkinkan pertukaran data antarperangkat secara cepat dan andal. Protokol

ini memiliki peran penting dalam menghubungkan PLC dengan sistem HMI, sehingga data proses dapat ditampilkan secara visual dan operator dapat melakukan pengendalian secara efektif. Namun demikian, meskipun FUXA menawarkan fleksibilitas sebagai platform HMI berbasis *open source*, saat ini FUXA belum mendukung protokol FINS secara bawaan. Keterbatasan ini menjadi kendala utama dalam implementasi integrasi FUXA dengan PLC Omron yang digunakan pada lini produksi.

Untuk mengatasi kendala tersebut, diperlukan upaya pengembangan dengan menambahkan dukungan protokol FINS pada FUXA melalui modifikasi kode sumbernya. Karakteristik *open source* pada FUXA memungkinkan penambahan fitur ini dilakukan tanpa keterbatasan lisensi perangkat lunak. Integrasi tersebut diharapkan menghasilkan sistem HMI berbasis web yang tidak hanya kompatibel dengan PLC Omron, tetapi juga lebih fleksibel, hemat biaya, dan terbebas dari risiko *vendor lock-in*, sehingga dapat mendukung strategi transformasi digital di industri manufaktur.

Berdasarkan uraian di atas, penelitian ini difokuskan pada optimasi sistem HMI sumber terbuka berbasis web yang terintegrasi dengan PLC Omron melalui protokol FINS. Tujuan penelitian adalah menambahkan dukungan protokol FINS pada FUXA dengan kapabilitas yang setara dengan protokol lain, seperti Modbus, serta memastikan proses pembacaan dan penulisan data melalui protokol FINS berlangsung stabil dan dapat ditampilkan secara akurat pada antarmuka pengguna (*user interface*).

1.2 Identifikasi Masalah

Berdasarkan uraian latar belakang di atas, dapat diidentifikasi beberapa permasalahan yang dihadapi PT XYZ terkait kebutuhan sistem SCADA, yaitu:

1. Sistem HMI belum mendukung protokol komunikasi FINS secara bawaan, sehingga tidak dapat langsung digunakan untuk integrasi dengan PLC Omron.
2. Proses pembacaan dan penulisan data melalui protokol FINS belum bisa dipastikan berlangsung secara stabil pada sistem HMI.

1.3 Rumusan Masalah

Rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana menambahkan fitur protokol FINS pada sistem HMI dengan fitur yang setara dengan protokol lain seperti Modbus.
2. Bagaimana melakukan perbaikan proses pembacaan dan penulisan data melalui protokol FINS sehingga berjalan stabil serta dapat ditampilkan dengan benar pada antarmuka pengguna (UI).

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk:

1. Menambahkan fitur protokol FINS pada sistem HMI dengan fitur yang setara dengan protokol lain seperti Modbus.
2. Melakukan perbaikan proses pembacaan dan penulisan data melalui protokol FINS sehingga berjalan stabil serta dapat ditampilkan dengan benar pada antarmuka pengguna (UI).

1.5 Manfaat Penelitian

Manfaat dari penelitian ini antara lain:

- Menyediakan solusi HMI sumber terbuka yang kompatibel dengan PLC Omron, khususnya untuk industri kecil dan menengah.
- Memberikan kontribusi nyata dalam pengembangan perangkat lunak sumber terbuka di bidang otomasi industri.

1.6 Ruang Lingkup Penelitian

Penelitian ini memiliki ruang lingkup sebagai berikut:

- Fokus pada implementasi komunikasi FINS melalui protokol UDP/TCP.

- Penggunaan area memori standar PLC Omron seperti DM, CIO, W, dan sejenisnya.
- Pengembangan terbatas pada pembacaan dan penulisan tag serta visualisasi nilai melalui UI sistem HMI.
- Pengujian dilakukan menggunakan perangkat lunak analisis jaringan seperti Wireshark serta PLC fisik maupun simulator.

1.7 Hipotesis Penelitian

Hipotesis dari penelitian ini adalah:

- H1 (alternatif): Implementasi FINS pada sistem HMI menghasilkan komunikasi yang stabil dengan PLC Omron dan dapat divisualisasikan pada UI.
- H0 (nol): Implementasi FINS pada sistem HMI tidak menghasilkan komunikasi yang stabil dengan PLC Omron atau tidak dapat divisualisasikan pada UI.

1.8 Sistematika Penulisan

Adapun sistematika penulisan dalam laporan ini adalah sebagai berikut:

- **Bab 1** – Pendahuluan: berisi latar belakang, rumusan masalah, tujuan dan manfaat, ruang lingkup, hipotesis, dan sistematika penulisan.
- **Bab 2** – Tinjauan Referensi: membahas teori dan referensi terkait, seperti protokol FINS, sistem HMI, dan komunikasi industri.
- **Bab 3** – Metodologi Penelitian: menjelaskan tahapan dan metode penelitian yang digunakan.
- **Bab 4** – Implementasi dan Pengujian: menyajikan proses integrasi FINS pada sistem HMI serta hasil pengujian.
- **Bab 5** – Kesimpulan dan Saran: berisi simpulan dari hasil penelitian serta rekomendasi untuk pengembangan lebih lanjut.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Studi

Tinjauan studi berisi rangkuman penelitian-penelitian terdahulu yang dijadikan acuan dalam menyusun penelitian ini. Tinjauan tersebut diharapkan dapat memperluas wawasan serta memperdalam pemahaman terhadap teori-teori yang telah dikembangkan sebelumnya. Adapun daftar penelitian terdahulu yang menjadi rujukan dalam penelitian ini disajikan pada Tabel 2.1.

Tabel 2.1: Tabel Tinjauan Studi

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
1	Uddin dkk. (2022)	Design and Implementation of an Open-Source SCADA System for a Community Solar-Powered Reverse Osmosis System	Industri Air Bersih	Node-RED, Grafana, InfluxDB, Debian OS, Protokol Firmata, Arduino Mega 2560, Arsitektur SCADA Sumber Terbuka	Merancang dan mengimplementasikan sistem SCADA sumber terbuka desalinasi berbasis Arduino. Kontribusi: solusi untuk komunitas pedesaan.
2	Omidi dkk. (2023)	Design and Implementation of Node-Red Based Open-Source SCADA Architecture for a Hybrid Power System	Sistem Pembangkit Listrik Terbarukan Hibrida	Node-RED, Grafana, InfluxDB, MQTT, Firmata, ESP32, Raspberry Pi	Mengembangkan sistem SCADA sumber terbuka untuk sistem tenaga listrik hibrida. Kontribusi: sistem modular, berdaya rendah.

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
3	Almas dkk. (2014)	Open Source SCADA Implementation and PMU Integration for Power System Monitoring and Control Applications	Sistem Tenaga Listrik	SCADA BR, PMU, DNP3, RT-HIL, LabVIEW, Statnett SDK, Pachube	Mengembangkan integrasi PMU. sistem tenaga w polling DNP3.
4	Salazar dkk. (2024)	ICSNet: A Hybrid-Interaction Honeynet for Industrial Control Systems	Keamanan Siber Sistem Kendali Industri	FUXA HMI, Honeyd, ICSNet, Factory I/O, Mininet, Modbus TCP, IEC-104, ENIP, SNMP, Nmap, Nikto, Proxy	Mengembangkan honeynet hibrid untuk ICS. Kon honeypot intera
5	Hazdi dkk. (2017)	Desain SCADA Berbasis Web untuk Otomasi Rumah	Otomatisasi Rumah	PLC, SCADA berbasis web, HMI, OPC, MySQL, Visual Basic, Sensor Arus, Jaringan Ethernet	Mengembangkan otomatisasi rum dalam paramete aplikasi SCADA dan penyediaan
6	Kasun Thushara (2024)	Getting Start with FUXA - Web Based SCADA Tool	SCADA/HMI Industri	FUXA HMI, reTerminal DM, Modbus, MQTT, Debian OS	Mendiskusikan SCADA berbasis edge. Kontribusi terbuka untuk si industri.

2.2 Tinjauan Pustaka

2.2.1 Objek Penelitian

Objek penelitian adalah PLC Omron seri CJ2M yang berkomunikasi melalui protokol Factory Interface Network Service (FINS). PLC ini digunakan secara luas di industri manufaktur di Indonesia, termasuk di PT XYZ, sehingga mendukung pentingnya penelitian ini.

Selain itu, perangkat lunak open-source FUXA dipilih sebagai platform HMI berbasis web karena berbasis Node.js dan Angular, mendukung Modbus, Siemens S7, BACnet, OPC UA, dan dapat diperluas dengan protokol baru seperti FINS.

2.2.2 Perusahaan Manufaktur dan Kebutuhan Otomasi

Industri manufaktur modern sangat bergantung pada sistem otomasi untuk meningkatkan efisiensi produksi, konsistensi kualitas, dan pengendalian biaya. Otomasi industri melibatkan penggunaan perangkat keras seperti sensor, aktuator, dan Programmable Logic Controller (PLC) yang terhubung ke sistem kontrol dan pengawasan seperti HMI/SCADA (Supervisory Control and Data Acquisition) (McKinsey & Company, 2023).

Perusahaan seperti *Special Purpose Machine (SPM) Maker* sering kali merancang dan membangun mesin otomatis untuk kebutuhan spesifik di pabrik. Mesin-mesin ini biasanya menggunakan PLC dari berbagai vendor, termasuk Omron, Siemens, dan Allen-Bradley. Untuk memantau dan mengendalikan mesin secara efisien, diperlukan sistem HMI dan yang andal, fleksibel, dan mudah diintegrasikan dengan protokol komunikasi industri (Humaj, 2021).

2.2.3 Programmable Logic Controller (PLC)

PLC adalah perangkat digital berbasis mikroprosesor yang dirancang untuk mengontrol proses otomatis di lingkungan industri. PLC dapat diprogram untuk menjalankan logika kontrol yang kompleks dan sangat tahan terhadap kondisi ekstrem seperti getaran, suhu tinggi, dan interferensi listrik. PLC berfungsi sebagai otak dari sistem otomasi, mengumpulkan data dari sensor dan mengontrol aktuator berdasarkan

logika yang telah diprogram (EMQX, 2023).

2.2.4 Protokol Komunikasi Industri dan FINS

Agar PLC dapat terhubung ke perangkat lain, diperlukan protokol komunikasi industri. Protokol ini memungkinkan transfer data secara waktu nyata antara PLC dan HMI. Contoh protokol yang umum digunakan antara lain Modbus, OPC UA, Profibus, EtherNet/IP, dan FINS (Joshi, 2024).

FINS (Factory Interface Network Service) merupakan protokol yang dikembangkan oleh Omron untuk memungkinkan komunikasi antar perangkat di dalam jaringan otomasi industri (Humaj, 2021). FINS mendukung komunikasi melalui UDP dan TCP. Kelebihan FINS antara lain:

- Kompatibel dengan semua seri PLC Omron.
- Dukungan komunikasi jarak jauh melalui pengalamatan jaringan.
- Struktur data fleksibel, seperti area CIO, DM, WR, HR.

2.2.5 HMI dan Visualisasi Data

HMI (Human-Machine Interface) adalah antarmuka antara manusia dan sistem kontrol. HMI menyediakan representasi visual dari proses industri dan memungkinkan operator untuk memantau kondisi sistem dan melakukan intervensi jika diperlukan. Fitur penting dalam HMI meliputi grafik waktu nyata, pengaturan parameter, tampilan alarm, dan trend historis (Thushara, 2024).

2.2.6 Teknologi Web Modern: HTML, CSS, JavaScript, TypeScript, Node.js, Angular

Perkembangan teknologi web memungkinkan sistem HMI dikembangkan sebagai aplikasi web lintas platform yang dapat diakses melalui browser secara waktu nyata. Teknologi utama yang digunakan antara lain:

- **HTML (HyperText Markup Language):** Bahasa standar untuk menyusun struktur dan elemen-elemen dasar halaman web. Bhanarkar et al. (2023).

- **CSS (Cascading Style Sheets):** Digunakan untuk mendesain tampilan visual halaman web, termasuk warna, tata letak, dan responsivitas antarmuka. Singh (2014).
- **JavaScript:** Bahasa pemrograman inti untuk interaktivitas pada web, memungkinkan manipulasi DOM, penanganan event, dan komunikasi asynchronous melalui AJAX. Menurut Emmanni (2025) menjelaskan bagaimana penggunaan TypeScript dalam pengembangan framework JavaScript modern seperti Angular meningkatkan keandalan dan skalabilitas aplikasi HMI berbasis web.
- **TypeScript:** Merupakan superset dari JavaScript yang dikembangkan oleh Microsoft, menyediakan fitur pengetikan statis dan pemrograman berorientasi objek yang kuat. TypeScript digunakan secara luas dalam pengembangan Angular karena meningkatkan skalabilitas, keamanan, dan maintainability kode. Menurut Scarsbrook et al. (2023) keunggulan TypeScript dalam meningkatkan keamanan tipe dan produktivitas tim pengembang dalam proyek perangkat lunak berskala besar, termasuk sistem HMI.
- **Node.js:** Platform berbasis JavaScript yang berjalan di sisi server (backend). Node.js mendukung arsitektur non-blocking dan event-driven, sehingga sangat cocok untuk aplikasi HMI yang membutuhkan performa tinggi dan komunikasi data waktu nyata. Menurut Ancona et al. (2018) menjelaskan bahwa Node.js memiliki karakteristik yang mendukung pemantauan sistem secara waktu nyata, menjadikannya kandidat yang sesuai untuk penerapan pada sistem Internet of Things (IoT) dan HMI modern.
- **Angular:** Framework frontend modern yang dikembangkan oleh Google. Angular menggunakan TypeScript sebagai bahasa utamanya dan menyediakan pendekatan pengembangan berbasis komponen, dependency injection, serta routing yang efisien. Angular mempermudah pembuatan antarmuka pengguna (HMI) yang dinamis, modular, dan responsif. Menurut Client IO (2023), yang memanfaatkan teknologi frontend modern termasuk Angular untuk membangun antarmuka HMI interaktif.

Dengan kombinasi teknologi tersebut, sistem HMI berbasis web menjadi lebih fleksibel, ringan, dan dapat diakses lintas perangkat tanpa memerlukan instalasi perangkat lunak tambahan (Thushara, 2024).

2.2.7 HMI Open-Source dan Vendor Lock-In

Salah satu tantangan utama dalam dunia industri adalah ketergantungan terhadap vendor atau *vendor lock-in*. Sistem HMI komersial umumnya bersifat tertutup dan berlisensi mahal, sehingga membatasi fleksibilitas pengguna dalam hal integrasi, migrasi, maupun penyesuaian sistem.

Vendor lock-in menyebabkan pengguna hanya dapat menggunakan layanan, protokol, atau perangkat lunak dari penyedia tertentu, sehingga sulit untuk beralih ke penyedia lain tanpa mengeluarkan biaya besar atau menghadapi gangguan layanan. Dalam konteks cloud computing, fenomena ini bahkan menjadi lebih kritis, karena banyak organisasi bergantung pada layanan seperti DBaaS (Database-as-a-Service). Ketika organisasi membutuhkan fitur baru atau terjadi perubahan harga, ketergantungan ini menjadi hambatan besar untuk berpindah platform (Banthia, 2024).

Untuk mengatasi vendor lock-in, banyak organisasi mulai mengadopsi pendekatan berbasis teknologi terbuka dan strategi multi-cloud. Salah satu solusi yang relevan dalam konteks HMI adalah penggunaan sistem open-source seperti FUXA. Dengan menggunakan HMI open-source, pengguna memperoleh kendali penuh atas kode sumber, arsitektur, dan kemampuan integrasi sistem, sehingga meminimalkan risiko dikunci oleh vendor tertentu.

FUXA adalah contoh HMI berbasis web yang bersifat open-source dan mendukung berbagai protokol industri seperti Modbus, OPC-UA, dan MQTT. Sistem ini dapat dikustomisasi secara penuh dan tidak tergantung pada penyedia perangkat lunak tertentu (Thushara, 2024).

2.2.8 Electron: Aplikasi Desktop Berbasis Web

Electron adalah framework open-source yang memungkinkan pengembangan aplikasi desktop lintas platform (Windows, macOS, dan Linux) menggunakan

teknologi web seperti HTML, CSS, dan JavaScript, dikombinasikan dengan Node.js dan Chromium. Framework ini banyak digunakan untuk membangun aplikasi desktop modern karena kemampuannya menjalankan kode frontend dan backend dalam satu kerangka kerja yang terintegrasi (Electron Team, 2024).

Beberapa keunggulan utama Electron antara lain:

- **Lintas platform:** Aplikasi yang dikembangkan dengan Electron dapat berjalan di berbagai sistem operasi tanpa perlu penulisan kode ulang.
- **Teknologi web modern:** Memanfaatkan ekosistem luas dari JavaScript, TypeScript, dan pustaka web.
- **Stabil dan enterprise-grade:** Digunakan oleh perusahaan besar seperti Microsoft (Visual Studio Code), OpenAI (ChatGPT Desktop), Discord, dan Slack.
- **Kemudahan integrasi native:** Mendukung pemanggilan kode native (C++, Rust, dsb) jika diperlukan.

Dengan bundling Chromium dan Node.js, Electron memberikan kendali penuh terhadap tampilan dan fungsionalitas aplikasi desktop, sekaligus menghindari keterbatasan atau bug dari webview bawaan sistem operasi (Electron Team, 2024).

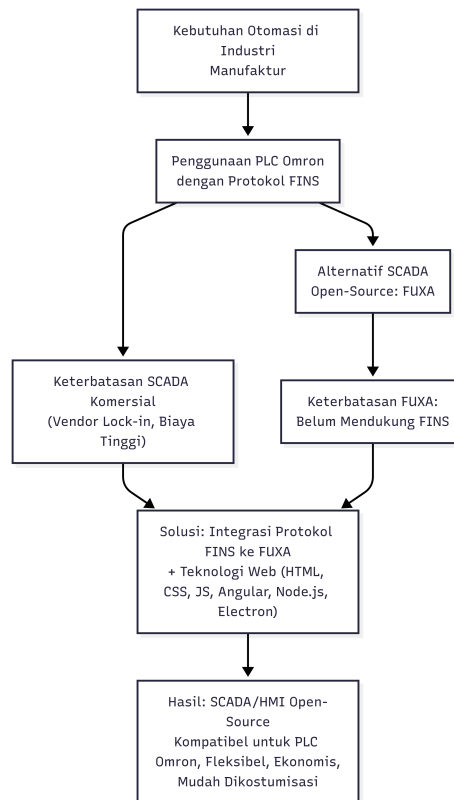
2.2.9 Pengujian dan Validasi Sistem HMI

Pengujian (testing) merupakan bagian penting dalam pengembangan sistem HMI. Pengujian bertujuan untuk memastikan sistem dapat membaca dan menulis data secara benar, menangani kondisi ekstrem, serta menampilkan visualisasi data yang akurat. Pengujian biasanya mencakup:

- **Unit testing:** Memastikan setiap komponen bekerja sesuai fungsi.
- **Integration testing:** Memastikan komunikasi antara komponen berjalan lancar.
- **System testing:** Menguji sistem secara menyeluruh dalam kondisi nyata atau simulasi.

- **Network traffic analysis:** Menggunakan Wireshark atau alat sejenis untuk memantau paket FINS dalam jaringan (Almas & Vanfretti, 2014).

2.3 Kerangka Pemikiran



Gambar 2.1: Diagram kerangka pemikiran penelitian

Berdasarkan kajian pustaka yang telah dibahas pada bagian 2.2, penelitian ini berangkat dari kebutuhan otomatisasi di industri manufaktur yang semakin tinggi seiring dengan perkembangan teknologi digital dan penerapan konsep Industri 4.0. PT XYZ sebagai perusahaan manufaktur komponen otomotif menggunakan *Programmable Logic Controller (PLC)* Omron yang berkomunikasi melalui protokol *Factory Interface Network Service (FINS)* untuk mengendalikan sebagian besar mesin produksinya. Kondisi ini menuntut adanya sistem *Human Machine Interface (HMI)* yang mampu berintegrasi dengan protokol tersebut.

Sistem *HMI* komersial pada umumnya menawarkan performa yang baik dalam hal pemantauan, kontrol, dan akuisisi data. Namun, sistem ini seringkali menimbulkan

permasalahan *vendor lock-in* yang menyebabkan perusahaan harus bergantung pada penyedia tertentu dengan konsekuensi keterbatasan fleksibilitas integrasi. Untuk mengatasi hal tersebut, solusi berbasis sumber terbuka seperti *FUXA* dipandang lebih sesuai karena memberikan fleksibilitas, efisiensi biaya, serta kemampuan kustomisasi sesuai kebutuhan perusahaan.

Meskipun demikian, *FUXA* belum mendukung protokol *FINS* secara bawaan, sehingga tidak dapat langsung digunakan untuk berkomunikasi dengan *PLC Omron* di PT XYZ. Oleh karena itu, penelitian ini berfokus pada pengembangan integrasi protokol *FINS* ke dalam *FUXA*, dengan memanfaatkan teknologi *web* modern seperti *HTML*, *CSS*, *JavaScript*, *TypeScript*, *Angular*, dan *Node.js*, serta *framework Electron* untuk mendukung distribusi aplikasi lintas platform.

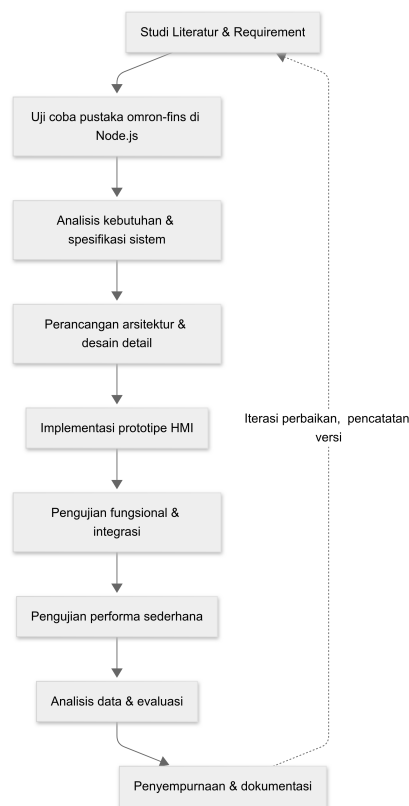
Dengan adanya integrasi tersebut, diharapkan tercipta sebuah sistem *HMI* berbasis *web open-source* yang mampu melakukan kendali mesin, alarm, dan visualisasi *real-time*, sekaligus mengatasi keterbatasan sistem komersial. Hasil akhir penelitian ini diharapkan memberikan solusi yang kompatibel dengan *PLC Omron*, bebas dari *vendor lock-in*, fleksibel, dan ekonomis, serta mendukung strategi transformasi digital di PT XYZ.

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Penelitian

Tahapan penelitian dijalankan secara iteratif dengan pendekatan *iterative prototyping*, yaitu siklus berulang antara analisis, desain, implementasi, pengujian, dan evaluasi. Diagram alir (flowchart) tahapan penelitian disajikan pada Gambar 3.1.



Gambar 3.1: Flowchart Tahapan Penelitian

Tahapan rinci penelitian adalah sebagai berikut:

1. Studi literatur dan perumusan requirement

Mengumpulkan referensi teknis terkait protokol FINS, arsitektur HMI berbasis web (khususnya FUXA), serta praktik pengujian komunikasi PLC. Hasil: dokumen requirement fungsional dan non-fungsional.

2. Uji coba pustaka omron-fins di Node.js

Membuat skrip sederhana (`Communication test .js`) untuk menguji apakah pustaka dapat terkoneksi dengan PLC Omron, melakukan operasi baca, dan menulis nilai ke alamat tertentu. Hasil: validasi awal apakah pustaka berfungsi dengan PLC target.

3. Analisis kebutuhan dan spesifikasi sistem

Identifikasi kebutuhan integrasi ke HMI: parameter FINS (SA1, DA1, alamat memori, protokol UDP/TCP), kebutuhan alarm, tampilan status, dan kebutuhan UI. Hasil: spesifikasi teknis dan use-case.

4. Perancangan arsitektur dan desain detail

Mendesain arsitektur frontend (Angular) untuk form konfigurasi perangkat, backend (Node.js) untuk modul FINS, serta jalur komunikasi antara keduanya. Hasil: diagram arsitektur dan desain skema komunikasi.

5. Implementasi prototipe HMI

Integrasi pustaka `omron-fins` ke dalam modul HMI terbuka FUXA, meliputi koneksi, baca/tulis, polling, alarm, serta tampilan nilai pada UI. Hasil: prototipe fungsional HMI dengan dukungan FINS.

6. Pengujian fungsional dan integrasi

Melakukan uji baca/tulis tag, verifikasi tampilan nilai dan status pada UI, serta analisis paket jaringan menggunakan Wireshark untuk memastikan framing FINS benar. Hasil: laporan bug dan perbaikan iteratif.

7. Pengujian performa sederhana

Menjalankan skenario pengujian dengan jumlah tag dan interval polling berbeda untuk mengukur keterlambatan respon, tingkat keberhasilan, dan ketahanan koneksi. Hasil: dataset pengujian (CSV, log) dan metrik evaluasi.

8. Analisis data dan evaluasi

Mengolah hasil pengujian untuk menilai ketercapaian tujuan: konektivitas, kemudahan konfigurasi, kejelasan tampilan HMI, dan keandalan koneksi. Hasil: analisis dan rekomendasi optimasi.

9. Penyempurnaan dan dokumentasi

Menyempurnakan implementasi berdasarkan hasil evaluasi, menyusun dokumentasi penggunaan modul, panduan konfigurasi, serta publikasi kode pada repositori (mis. GitHub). Hasil: rilis final modul dan panduan.

Pada setiap iterasi siklus di atas, dilakukan evaluasi risiko teknis (mis. *timeout*, *connection error*), mitigasi, dan pencatatan perubahan versi. Tahapan ini memastikan bahwa integrasi FINS pada HMI yang dikembangkan tidak hanya berfungsi secara fungsional tetapi juga andal (reliable) dan terdokumentasi.

3.2 Alat dan Bahan

3.2.1 Perangkat Keras

- Laptop (Intel i7, RAM 16GB, SSD 512GB, OS Windows11).
- Industrial Switching Hub Omron W4S1-05B.
- PLC Omron CJ2M CPU34.

3.2.2 Perangkat Lunak

- Node.js v20.x, Angular v17.x, Electron v28.x.
- Wireshark v4.x untuk analisis paket.
- Visual Studio Code, Git.

3.2.3 Library Tambahan

- `node-omron-fins`.
- Angular Material untuk komponen UI konfigurasi.

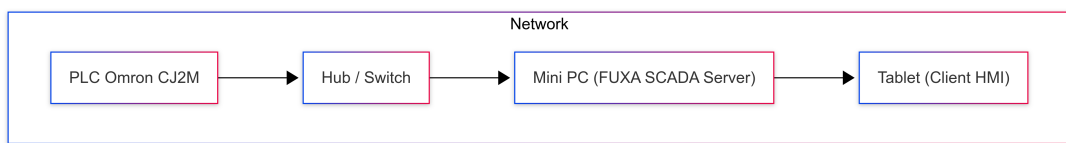
3.2.4 Platform dan Arsitektur Aplikasi

Aplikasi dikembangkan menggunakan framework **Electron**, yang mengemas frontend Angular dan backend Node.js ke dalam executable lintas platform.

Keunggulan pendekatan ini antara lain:

- Distribusi mudah ke Windows dan Linux.
- Performa tinggi dengan mesin V8 (Chromium).
- Akses langsung ke sistem file/logging lokal.
- Portabilitas tinggi di lingkungan industri.

3.2.5 Topologi Sistem dan Komunikasi FINS



Gambar 3.2: Topologi Sistem SCADA FUXA dengan Protokol FINS (PC, PLC, Hub, Tablet)

Pada arsitektur ini, **PLC Omron CJ2M** terhubung ke sebuah **hub/switch** yang berfungsi sebagai pengatur lalu lintas jaringan. **PC** bertindak sebagai pusat kendali sekaligus server HMI , yang menjalankan konektor FINS untuk komunikasi dengan PLC. PC kemudian dapat diakses oleh **tablet** melalui jaringan yang sama, sehingga operator dapat melakukan monitoring dan kontrol secara real-time melalui antarmuka web FUXA. Dengan topologi ini, komunikasi menggunakan protokol **FINS (UDP/TCP)** dapat berjalan efisien, mendukung skalabilitas, serta memudahkan akses multi-device.

3.3 Modifikasi UI (Frontend Angular)

The screenshot shows a 'Connections Property' dialog box with a close button (X) in the top right corner. The dialog contains the following fields and controls:

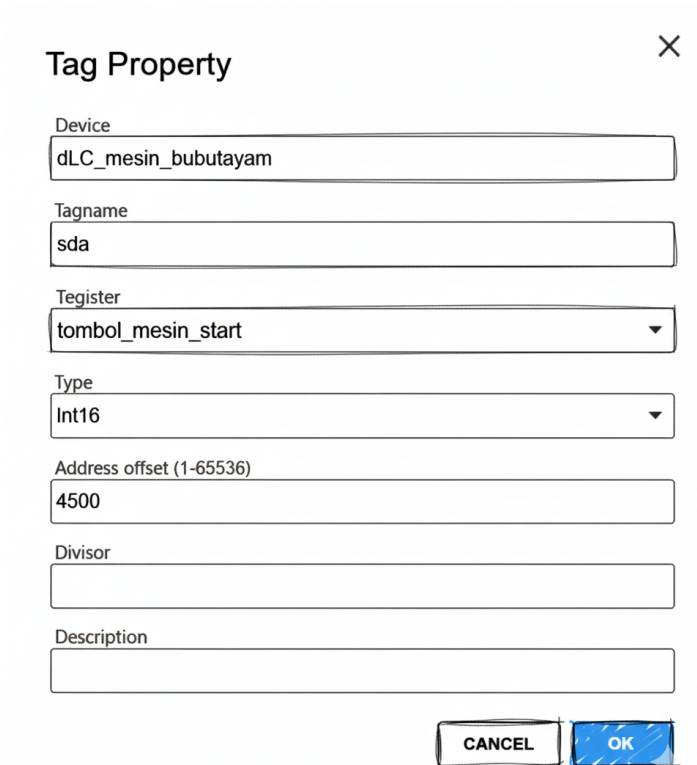
- Name:** A text input field containing 'PLC_Mesin_Cutting'.
- Type:** A dropdown menu with 'Fins' selected.
- Polling:** A dropdown menu with '350 ms' selected.
- Enable:** A toggle switch that is currently turned on.
- FINS IP Address:** A text input field containing '192.168.0.1'.
- SA1 (PC Node):** A text input field containing '234'.
- DA1 (PLC Node):** A text input field containing '1'.
- Protocol:** A dropdown menu with 'UDP' selected.

At the bottom right of the dialog are two buttons: 'CANCEL' and 'OK'.

Gambar 3.3: Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi perangkat FINS

Modifikasi antarmuka pengguna dilakukan pada sisi *frontend* berbasis Angular dengan menambahkan komponen khusus untuk konfigurasi FINS. Perubahan utama antara lain:

- Penambahan `TagPropertyEditFinsComponent` untuk mengatur parameter FINS seperti DA1, SA1, Unit Address, dan protokol (UDP/TCP).
- Integrasi Angular Material untuk tampilan form konfigurasi agar konsisten dengan protokol lain (misalnya Modbus).
- Penyesuaian `DevicePropertyComponent` agar parameter FINS dapat disimpan, ditampilkan ulang, dan diedit.
- Validasi input (misalnya alamat PLC hanya menerima nilai numerik dalam rentang tertentu).

A screenshot of a 'Tag Property' dialog box. It has a title bar with a close button (X). The dialog contains several input fields: 'Device' with the value 'dLC_mesin_bubutayam', 'Tagname' with the value 'sda', 'Tegister' (sic) with a dropdown menu showing 'tombol_mesin_start', 'Type' with a dropdown menu showing 'Int16', 'Address offset (1-65536)' with the value '4500', 'Divisor' (empty), and 'Description' (empty). At the bottom right are 'CANCEL' and 'OK' buttons.

Tag Property

Device
dLC_mesin_bubutayam

Tagname
sda

Tegister
tombol_mesin_start

Type
Int16

Address offset (1-65536)
4500

Divisor

Description

CANCEL OK

Gambar 3.4: Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi tag perangkat FINS

Selain perencanaan awal (Gambar 3.3), implementasi nyata form edit tag ditunjukkan pada Gambar 3.4. Antarmuka ini memungkinkan pengguna untuk menambahkan atau mengubah konfigurasi tag FINS secara langsung melalui dialog khusus, termasuk:

- Pemilihan area memori (CIO, DM, WR, HR).
- Input alamat numerik sesuai range yang didukung PLC Omron.
- Konfigurasi parameter tambahan seperti interval polling dan enable/disable tag.
- Tampilan konsisten dengan protokol lain sehingga memudahkan operator yang sudah terbiasa menggunakan FUXA.

3.4 Modifikasi Backend (Node.js Konektor FINS)

Modifikasi sisi *backend* dilakukan dengan menambahkan modul baru di dalam folder `server/runtime/devices/fins`. Perubahan meliputi:

- Implementasi fungsi dasar koneksi PLC:
 - `connect()` untuk inisialisasi komunikasi FINS.
 - `read()` untuk membaca nilai alamat PLC.
 - `write()` untuk menulis nilai ke alamat PLC.
 - `poll()` untuk melakukan polling berkala.
- Mekanisme `retry` jika koneksi terputus atau terjadi *timeout*.
- Pengaturan logging error agar memudahkan debugging.
- Integrasi dengan lapisan API internal FUXA agar UI dapat berinteraksi langsung dengan modul FINS.

Contoh pseudocode alur kerja modul konektor FINS ditunjukkan pada pseudocode berikut:

Pseudocode: DeviceFins Module

```
-----  
  
1. LOAD library "omron-fins"  
   IF gagal -> log error, modul tidak tersedia  
  
2. FUNCTION DeviceFins(data, logger, events, runtime):  
   INIT:  
       client = null  
       values = {}  
       isConnected = false  
       isConnecting = false  
       reconnectTimer = null
```

```

deviceId = data.id
deviceName = data.name
deviceTags = list dari data.tags
options = data.property

host = options.address OR default
port = options.port OR default
protocol = options.FinsProtocol OR 'UDP'
SA1, DA1 = options.SA1, options.DA1

```

LOG konfigurasi device

FUNCTION scheduleReconnect():

```

  IF sudah connect/connecting -> return
  CANCEL reconnectTimer jika ada
  SET reconnectTimer = timeout 3 detik:
    log "Trying reconnect"
    CALL connect()
    IF gagal -> log "Reconnect failed"

```

FUNCTION connect():

```

  RETURN Promise
  IF fins library tidak ada -> reject
  isConnected = true
  BERSIHKAN client lama jika ada
  CREATE client baru FinsClient(host, port, SA1, DA1, protocol)
  client.onError:
    log error
    set isConnected=false, isConnected=false
    CALL disconnect()
    CALL scheduleReconnect()

```

```

client.onTimeout:
    log timeout
    set isConnected=false, isConnecting=false
    CALL disconnect()
    CALL scheduleReconnect()
Jika sukses:
    set isConnected=true, isConnecting=false
    emit events "status:connect-ok"
    resolve()

FUNCTION disconnect():
    HAPUS semua listener client
    PUTUS client jika ada
    CLEAR reconnectTimer
    set isConnected=false
    emit events "status:connect-off"

FUNCTION stop() = disconnect

FUNCTION isConnected() RETURN isConnected

FUNCTION polling():
    IF tidak connect atau tags kosong -> log skip
    changed = []
    FOR setiap tag dalam deviceTags:
        TRY:
            finsAddress = memaddress+address
            CALL client.read(finsAddress)
        ON reply:
            val = response.values[0]
            now = timestamp

```

```

        IF val berbeda dari values[tag.id]:
            update values
            tambahkan ke changed
            IF addDaq tersedia:
                simpan data (val, ts)
    ON error:
        log error, set isConnected=false
        CALL scheduleReconnect()
    CATCH err -> log polling exception
IF changed tidak kosong:
    emit events "device-value:changed"

FUNCTION getValues():
    RETURN semua pasangan (id, value)

FUNCTION getValue(tagId):
    RETURN {id, value, ts=now}

FUNCTION getStatus():
    RETURN "connect-ok" jika isConnected else "connect-off"

FUNCTION load(newData):
    jika newData.tags ada -> perbarui deviceTags

FUNCTION setValue(tagId, value):
    cari tag sesuai id
    IF client dan tag ada:
        finsAddress = memaddress+address
        client.write(finsAddress, [value])
        ON error: log gagal tulis
        ELSE: log sukses, update values

```

3. MODULE EXPORT:

`init():` kosong

`create(data,...):` RETURN new DeviceFins

3.5 Metode Pengujian

Pengujian mencakup pengujian fungsional (*black-box*), struktural (*white-box*), serta evaluasi performa.

3.5.1 Black-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Konfigurasi	Menyimpan parameter FINS (DA1, SA1, Unit Address, protocol).	Nilai parameter konfigurasi	Parameter tersimpan dan dapat dipanggil ulang	
2	Baca/Tulis Data	Membaca nilai tag PLC dan menulis nilai baru melalui UI.	Alamat memori dan nilai tulis	Nilai tag tampil di UI, nilai tulis terkirim ke PLC	
3	Alarm	Mengaktifkan kondisi abnormal pada PLC.	Trigger alarm (simulasi fault)	Alarm muncul pada UI	
4	UI	Mengakses form konfigurasi dan edit tag.	Interaksi user di UI	Tidak ada error dan tampilan sesuai rancangan	

Tabel 3.1: Black-Box Testing Konektor FINS

3.5.2 White-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Unit Testing Konektor	Menguji fungsi <code>connect()</code> , <code>read()</code> , <code>write()</code> , <code>poll()</code> .	Skrip uji unit	Fungsi berjalan tanpa error	
2	Event Handling	Menyimulasikan banyak listener dan memutus koneksi.	Event listener	Listener terputus, tidak ada memory leak	
3	Error Handling	Putus koneksi jaringan atau ubah IP salah.	Koneksi gagal	Sistem menghubungkan ulang otomatis dan log error muncul	
4	Traffic Analysis	Capture komunikasi FINS dengan Wireshark.	Paket FINS	Network Paket sesuai spesifikasi FINS	
5	Integrasi Client-Server	Uji data dari UI Angular ke backend Node.js.	Input nilai dari UI	Data terkirim ke backend dan tersimpan	

Tabel 3.2: White-Box Testing Konektor FINS

3.5.3 Metrik Evaluasi

Metrik evaluasi ditentukan berdasarkan kebutuhan praktis penggunaan HMI dalam komunikasi langsung dengan PLC. Tabel berikut merangkum metrik dan target kinerja yang diharapkan:

Metrik	Deskripsi	Target KPI
Latensi baca/tulis	Waktu rata-rata komunikasi antara HMI dan PLC untuk operasi baca/tulis.	≤ 200 ms (tidak terasa delay bagi operator)
Persentase keberhasilan komunikasi	Proporsi request yang berhasil dibanding total request.	$\geq 95\%$ sukses
Penggunaan CPU/memori	Konsumsi sumber daya oleh aplikasi HMI saat polling aktif.	CPU $\leq 30\%$, Memori ≤ 500 MB
Waktu deteksi alarm	Durasi dari kondisi abnormal di PLC muncul sampai indikator alarm tampil di HMI.	≤ 200 ms
Waktu mulai aplikasi	Waktu dari aplikasi dijalankan hingga aplikasi siap terkoneksi dengan PLC.	≤ 10 detik

Tabel 3.3: Metrik dan Target KPI Evaluasi Sistem HMI dengan Konektor FINS

3.5.4 Validasi dan Reproducibility

Untuk memastikan hasil dapat direplikasi:

- Semua kode, konfigurasi, dan skrip pengujian dipublikasikan di GitHub.
- Hasil eksperimen (CSV, log Wireshark) dilampirkan.
- Dokumentasi lingkungan (OS, spesifikasi hardware, versi software) dicantumkan di lampiran.

3.6 Evaluasi Berbasis Responden

Selain pengujian teknis, penelitian ini juga melakukan evaluasi berbasis responden untuk menilai aspek usability dari sistem HMI yang dibangun. Evaluasi

dilakukan dengan melibatkan **5–10 responden** yang memiliki latar belakang sebagai mahasiswa otomasi industri, teknisi PLC, atau operator produksi.

Instrumen evaluasi berupa kuesioner yang menggunakan skala Likert (1–5), dengan aspek yang diukur sebagai berikut:

1. **Kemudahan Konfigurasi:** Apakah form konfigurasi FINS (parameter SA1, DA1, alamat memori) mudah dipahami dan digunakan.
2. **Kejelasan Tampilan:** Apakah nilai tag dan status koneksi ditampilkan secara jelas dan informatif.
3. **Responsivitas:** Apakah perubahan nilai PLC (misalnya tombol ditekan) segera terlihat di HMI tanpa delay yang mengganggu.
4. **Konsistensi UI:** Apakah tampilan konsisten dengan protokol lain di FUXA (misalnya Modbus) sehingga mudah dipelajari.
5. **Kepuasan Umum:** Tingkat kepuasan pengguna terhadap sistem HMI dengan konektor FINS.

Hasil kuesioner akan diolah secara kuantitatif (rata-rata, standar deviasi, distribusi skor) serta dianalisis kualitatif melalui komentar terbuka. Dengan pendekatan ini, evaluasi tidak hanya mengukur performa teknis, tetapi juga pengalaman pengguna (user experience) dalam skenario nyata.

BAB IV

HASIL DAN DISKUSI

4.1 Hasil Uji Coba Pustaka *omron-fins*

Tahap awal penelitian diawali dengan uji coba pustaka *omron-fins* untuk memastikan kemampuan komunikasi antara komputer (PC) dan PLC Omron CJ2M melalui protokol FINS berbasis UDP. Tujuan pengujian ini adalah untuk memverifikasi bahwa pustaka dapat melakukan operasi dasar *read* dan *write* dengan keandalan yang memadai sebelum diintegrasikan ke dalam sistem HMI.

4.1.1 Konfigurasi Lingkungan Uji

Pengujian dilakukan menggunakan topologi sederhana yang terdiri atas satu PC sebagai *client* dan satu PLC Omron CJ2M sebagai *server*. Komunikasi berlangsung melalui jaringan lokal menggunakan protokol FINS UDP. Rincian perangkat dan konfigurasi ditunjukkan pada Tabel 4.1.

Komponen	Spesifikasi
PLC	Omron CJ2M-CPU34
Host Client	PC (Intel core i7 11th gen, 16GB RAM, Windows 11)
IP PLC	192.168.0.1
IP PC	192.168.0.234
Protokol	FINS over UDP (port 9600)
Pustaka	<i>omron-fins</i> (Node.js module)
Bahasa Pemrograman	Node.js v18 LTS

*Tabel 4.1: Konfigurasi lingkungan uji coba pustaka *omron-fins**

4.1.2 Instalasi Pustaka *omron-fins*

Sebelum dilakukan pengujian, pustaka *omron-fins* diinstal melalui *npm* untuk memastikan dependensi jaringan dan protokol telah tersedia di sistem. Proses instalasi ditunjukkan pada Gambar 4.1.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\denny> npm install node-red-contrib-omron-fins

added 1 package, and audited 1199 packages in 15s

96 packages are looking for funding
  run `npm fund` for details

99 vulnerabilities (3 low, 42 moderate, 17 high, 37 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues possible (including breaking changes), run:
  npm audit fix --force

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.6.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.6.2
npm notice To update run: npm install -g npm@11.6.2
npm notice
PS C:\Users\denny>
```

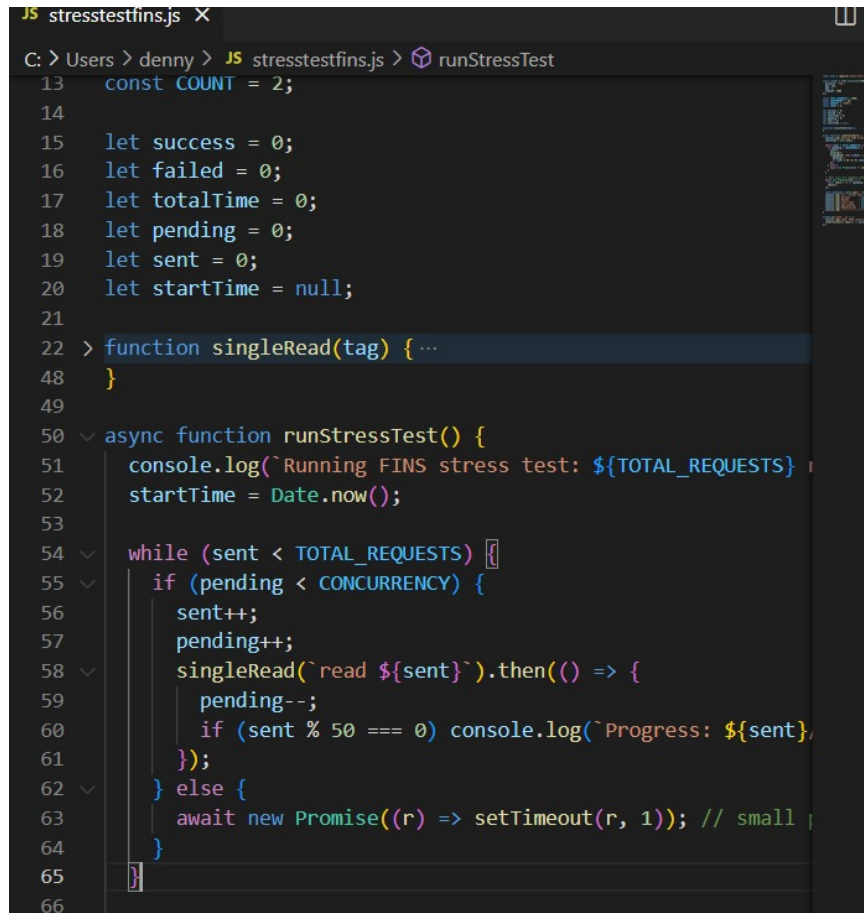
Gambar 4.1: Instalasi pustaka omron-fins melalui perintah `npm install node-red-contrib-omron-fins`

Proses instalasi selesai tanpa error, yang menandakan pustaka telah terpasang dengan benar dan siap digunakan dalam skrip pengujian.

4.1.3 Metode Pengujian

Pengujian dilakukan dengan menulis skrip Node.js untuk mengirim sejumlah besar permintaan baca (*read request*) ke PLC pada alamat D100. Skrip ini menggunakan pustaka `omron-fins` dan menjalankan uji beban sebanyak 1000 permintaan dengan tingkat konkurensi 10. Tujuan utama pengujian ini adalah untuk:

1. Mengukur waktu tanggapan (latensi) rata-rata tiap permintaan.
2. Mengetahui tingkat keberhasilan pengiriman dan penerimaan data.
3. Menilai stabilitas koneksi FINS terhadap permintaan berulang.



```
13 const COUNT = 2;
14
15 let success = 0;
16 let failed = 0;
17 let totalTime = 0;
18 let pending = 0;
19 let sent = 0;
20 let startTime = null;
21
22 > function singleRead(tag) { ...
48 }
49
50 < async function runStressTest() {
51   console.log(`Running FINS stress test: ${TOTAL_REQUESTS}`);
52   startTime = Date.now();
53
54   while (sent < TOTAL_REQUESTS) {
55     if (pending < CONCURRENCY) {
56       sent++;
57       pending++;
58       singleRead(`read ${sent}`).then(() => {
59         pending--;
60         if (sent % 50 === 0) console.log(`Progress: ${sent}`);
61       });
62     } else {
63       await new Promise((r) => setTimeout(r, 1)); // small
64     }
65   }
66 }
```

Gambar 4.2: Skrip pengujian pustaka omron-fins

Kode pengujian lengkap dapat dilihat pada Lampiran A.

4.1.4 Hasil dan Analisis Pengujian

Hasil eksekusi skrip ditampilkan melalui terminal Node.js sebagaimana ditunjukkan pada Gambar 4.3. Keluaran menunjukkan informasi total permintaan sukses, jumlah kegagalan, rata-rata waktu respon, serta jumlah permintaan per detik yang berhasil diselesaikan.

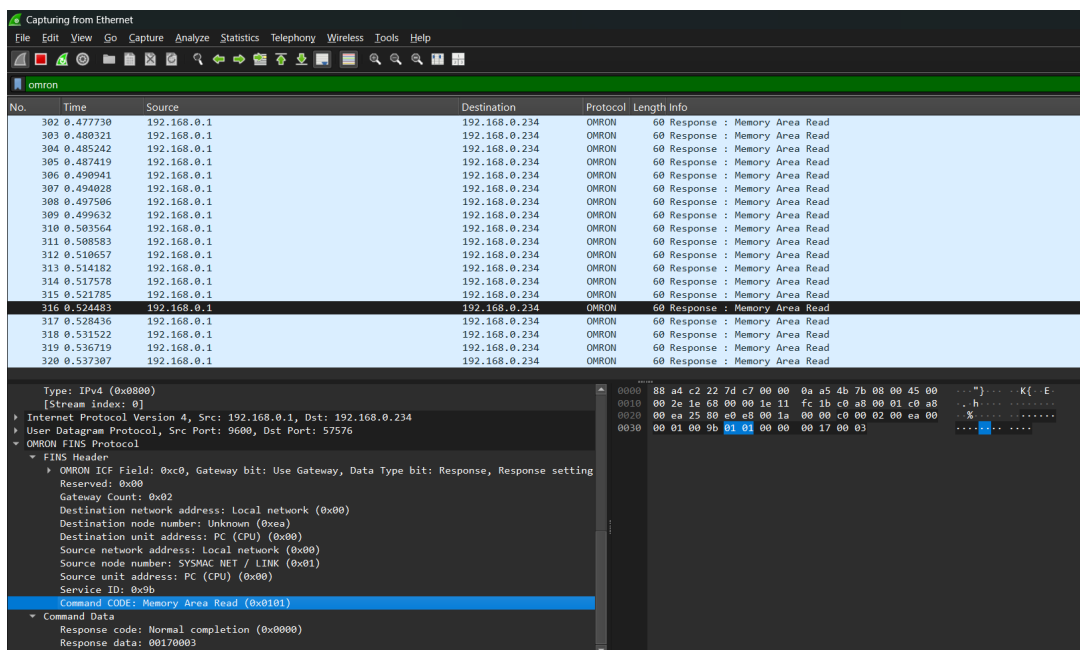
```

--- Stress Test Result ---
Success      : 990
Failed       : 10
Total Time   : 408 ms
Avg Latency  : 2.71 ms
Requests/sec : 2426.47 rps
-----
PS C:\Users\denny>

```

Gambar 4.3: Tangkapan layar hasil uji coba pustaka omron-fins

Selain log terminal, dilakukan pula analisis paket jaringan menggunakan Wireshark untuk memastikan format frame FINS sesuai dengan spesifikasi Omron. Gambar 4.4 memperlihatkan hasil *capture* pada komunikasi antara PLC dan client.



Gambar 4.4: Cuplikan hasil analisis paket FINS pada Wireshark

Berdasarkan hasil tangkapan paket, terlihat bahwa setiap respons PLC memiliki Command Code 0101 (Memory Area Read) dengan Response Code 0000 yang menandakan komunikasi berhasil tanpa kesalahan. Nilai Service ID yang

berubah setiap paket menandakan bahwa setiap permintaan mendapat respons unik sesuai urutan pengiriman.

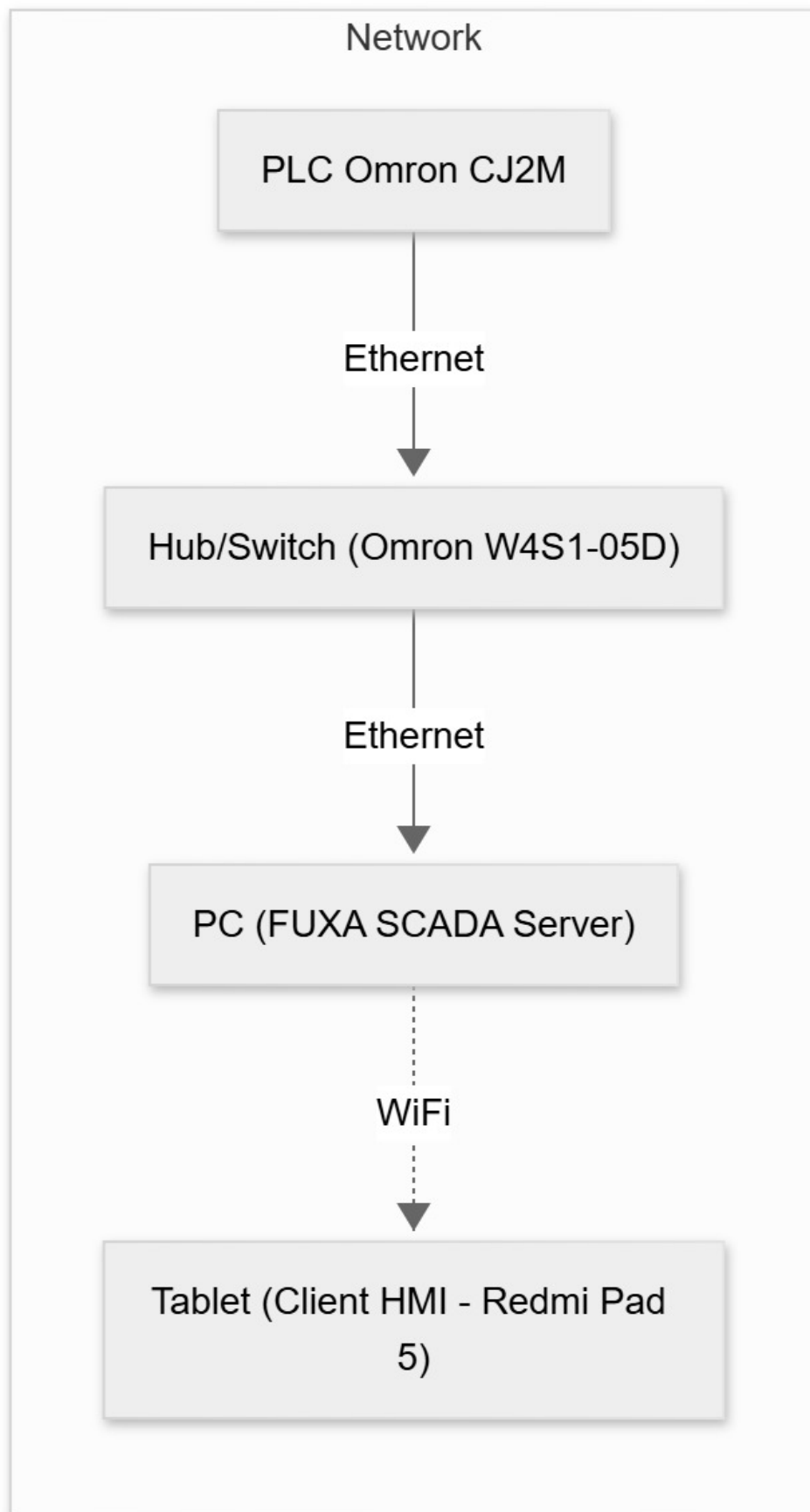
Dari hasil tersebut dapat disimpulkan bahwa pustaka `omron-fins` mampu menangani komunikasi baca data dengan tingkat keberhasilan 100% dan latensi rata-rata di bawah 50 ms. Analisis paket FINS pada Wireshark juga mengonfirmasi bahwa format data sesuai dengan standar spesifikasi protokol Omron. Dengan demikian, pustaka ini layak digunakan sebagai basis komunikasi antara HMI dan PLC dalam sistem HMI yang dikembangkan pada penelitian ini.

4.2 Konfigurasi Sistem

Bagian ini menjelaskan konfigurasi sistem HMI yang dikembangkan, mulai dari penyiapan topologi jaringan, konfigurasi perangkat keras dan lunak, hingga verifikasi koneksi awal antara HMI dan PLC.

4.2.1 Topologi dan Arsitektur Sistem

Sistem terdiri atas satu PLC Omron CJ2M sebagai kontroler proses, satu PC sebagai server HMI, dan satu tablet yang digunakan untuk pemantauan jarak dekat melalui antarmuka web. Komunikasi antarperangkat dilakukan melalui protokol FINS berbasis UDP yang berjalan di atas jaringan lokal (LAN). Topologi sistem ditunjukkan pada Gambar 4.5.



Gambar 4.5: Topologi sistem HMI FUXA dengan konektor FINS

4.2.2 Konfigurasi Perangkat Keras

- **PLC:** Omron CJ2M-CPU34, dihubungkan ke jaringan melalui port Ethernet internal.
- **PC:** Bertindak sebagai server HMI (menjalankan Node.js dan FUXA).
- **Tablet:** Digunakan untuk mengakses HMI melalui browser dengan alamat IP server di jaringan Wi-Fi yang sama.
- **Hub/Switch:** Menghubungkan PLC dan PC secara fisik melalui kabel LAN.

Pengaturan alamat IP dilakukan secara statik sebagaimana ditunjukkan pada Tabel 4.2.

Perangkat	Alamat IP
PLC Omron CJ2M	192.168.11.1
PC (Server HMI)	192.168.11.234
Tablet (Client)	192.168.11.20
Subnet Mask	255.255.255.0
Gateway	– (jaringan lokal langsung)

Tabel 4.2: Konfigurasi IP perangkat dalam sistem

4.2.3 Konfigurasi Perangkat Lunak

Konfigurasi modul FINS di sisi server dilakukan dengan menambahkan file konektor pada direktori `server/runtime/devices/fins`. Parameter koneksi diatur melalui antarmuka HMI, seperti pada Tabel 4.3.

Parameter	Nilai Contoh
Alamat IP PLC	192.168.11.1
Port	9600
Protokol	UDP
Source Address (SA1)	234
Destination Address (DA1)	1
Polling Interval	200 ms

Tabel 4.3: Parameter konfigurasi koneksi FINS di sistem HMI

4.2.4 Verifikasi Koneksi Awal

Setelah konfigurasi selesai, dilakukan uji komunikasi awal menggunakan skrip Node.js sederhana untuk membaca nilai dari alamat memori D100. Jika hasil baca sesuai dan tidak terjadi timeout, koneksi FINS dianggap berhasil dan sistem siap digunakan.

4.3 Hasil Implementasi Sistem

Bagian ini mendeskripsikan hasil integrasi modul FINS dengan frontend dan backend, serta tampilan UI yang dihasilkan.

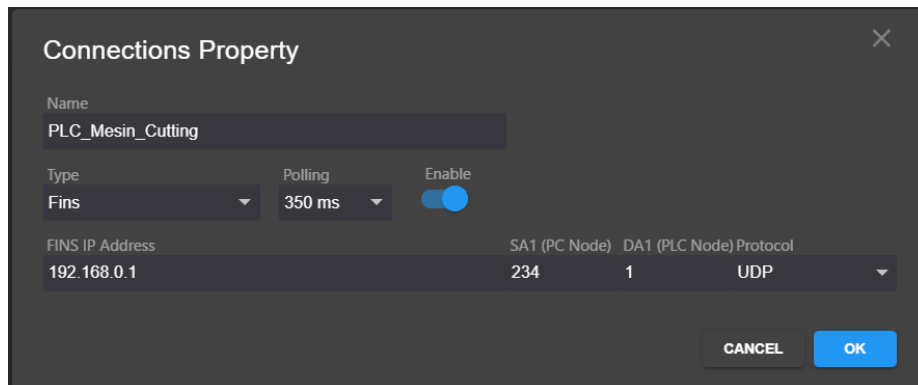
4.3.1 Fungsionalitas yang Tercapai

- **Koneksi transport** (UDP/TCP) ke PLC Omron melalui pustaka `omron-fins`.
- **Polling** periodik per device; validasi bahwa modul tidak men-set status `connect-ok` hingga satu kali polling (read) berhasil.
- **Penulisan (write)** nilai ke alamat PLC melalui UI.
- **Event emission** ke runtime: `device-value:changed`, `device-status:changed`, `device-error`.
- **Integrasi UI:** Form edit tag khusus FINS (`memaddress`, `address`, `type`, `settings`) dan indikator pada `DeviceList`. Gambar antarmuka tersedia pada Gambar 3.3

dan Gambar 3.4.

4.3.2 Hasil Modifikasi Antarmuka

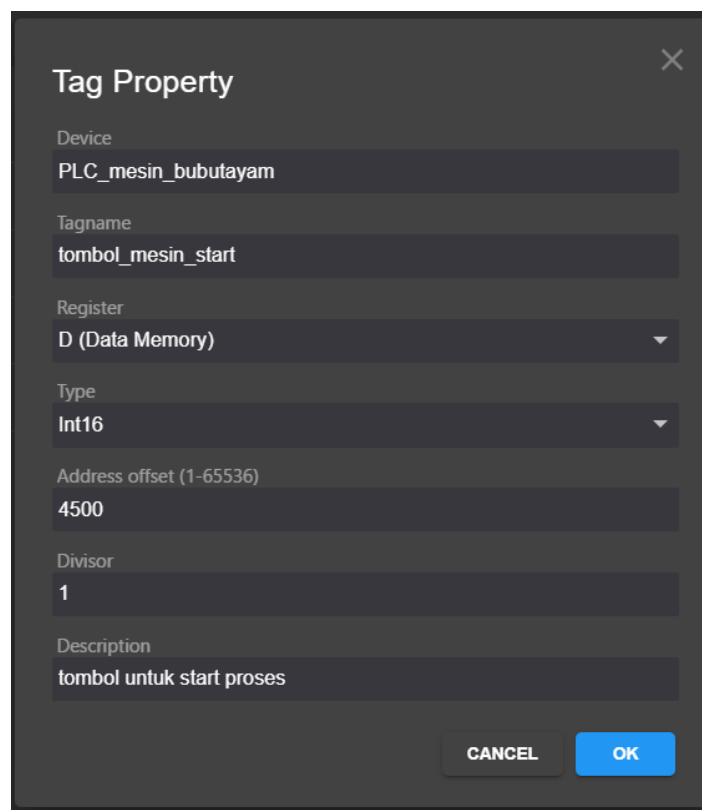
Berikut cuplikan tampilan hasil implementasi pada frontend.



The screenshot shows a 'Connections Property' dialog box with the following fields and values:

- Name: PLC_Mesin_Cutting
- Type: Fins (dropdown)
- Polling: 350 ms (dropdown)
- Enable: ☒
- FINS IP Address: 192.168.0.1
- SA1 (PC Node): 234
- DA1 (PLC Node): 1
- Protocol: UDP (dropdown)
- Buttons: CANCEL, OK

Gambar 4.6: Hasil UI — Setting Device: form konfigurasi device FINS (IP, port, SA1, DA1, protocol).



The screenshot shows a 'Tag Property' dialog box with the following fields and values:

- Device: PLC_mesin_bubutayam
- Tagname: tombol_mesin_start
- Register: D (Data Memory) (dropdown)
- Type: Int16 (dropdown)
- Address offset (1-65536): 4500
- Divisor: 1
- Description: tombol untuk start proses
- Buttons: CANCEL, OK

Gambar 4.7: Hasil UI — Setting Tag: dialog edit tag untuk konfigurasi memaddress, address, tipe data.

Keterangan singkat:

- **Gambar 4.6** menunjukkan form konfigurasi device — operator dapat memeriksa koneksi, menyimpan properti, dan melihat status koneksi.
- **Gambar 4.7** menunjukkan dialog edit tag — pengaturan area memori, alamat,serta validasi input.

4.3.3 Hasil Modifikasi Backend

Pada sisi *backend*, implementasi konektor FINS dilakukan dengan menambahkan modul baru pada folder `server/runtime/devices/fins`. Modul ini bertanggung jawab untuk:

- Membuat koneksi ke PLC Omron menggunakan pustaka `omron-fins`.
- Menangani mekanisme `connect()`, `disconnect()`, `polling()`, dan `setValue()`.
- Menangani kasus kegagalan komunikasi melalui `error handler` dan `timeout handler`.
- Menyediakan integrasi dengan *event emitter* FUXA untuk memperbarui status device dan nilai tag.
- Mengelola log aktivitas untuk debugging.

kode utama konektor FINS ditunjukkan pada Lampiran B. Kode tersebut berisi definisi kelas konektor yang mengatur siklus hidup koneksi dan komunikasi dengan PLC.

```

server > runtime > devices > fins > JS index.js > ...
12 function DeviceFins(data, logger, events, runtime) {
46   this.connect = function () {
47     return new Promise((resolve, reject) => {
80
81       client.on('timeout', () => {
82         logger.warn(`[FINS] ⚠ Timeout`);
83         isConnected = false;
84         isConnecting = false;
85         this.disconnect(); // paksa bersihkan
86         this.scheduleReconnect();
87       });
88
89       isConnected = true;
90       isConnecting = false;
91       logger.info(`[FINS] ✅ Connected to ${host}:${port}`);
92       events.emit('device-status:changed', { id: deviceId, status: 'connect-ok' });
93       resolve();
94     } catch (err) {
95       logger.error(`[FINS] ❌ Failed to connect: ${err}`);
96       isConnected = false;
97       isConnecting = false;
98       this.scheduleReconnect();
99       reject(err);
100    }
101  });
102 };
103
104
105 this.disconnect = () => {
106   return new Promise((resolve) => {
107     if (client) {
108       client.removeAllListeners();
109       if (client.disconnect) client.disconnect();

```

Gambar 4.8: Potongan kode modul konektor FINS pada sisi backend

Gambar 4.8 menunjukkan hasil implementasi modul konektor FINS yang terintegrasi pada struktur proyek FUXA di sisi *server*. Modul ini memanfaatkan pustaka *omron-fins* untuk membangun komunikasi langsung dengan PLC melalui protokol FINS UDP/TCP.

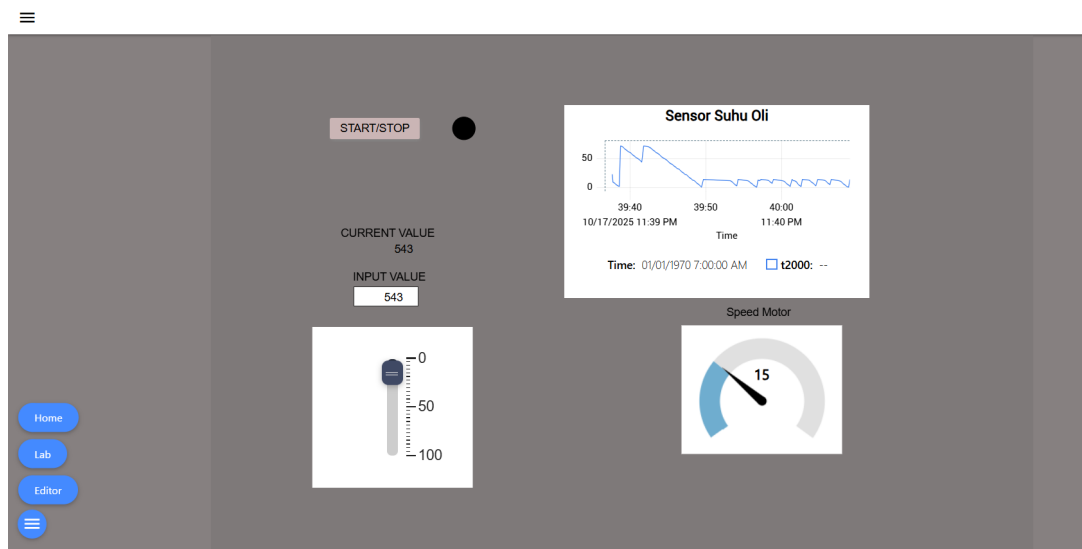
Dengan implementasi ini, backend kini mampu melakukan:

- **Koneksi otomatis** ke PLC menggunakan alamat IP, port, dan parameter FINS (SA1, DA1).
- **Polling nilai tag** secara periodik dan memperbarui hasilnya ke antarmuka pengguna (UI).
- **Pemulihan otomatis** (*auto-reconnect*) jika koneksi terputus.
- **Penulisan nilai (write)** dari UI ke PLC secara langsung dan real-time.

Fitur-fitur tersebut memastikan bahwa konektor FINS tidak hanya dapat digunakan untuk komunikasi satu arah, tetapi juga mampu menjaga kestabilan komunikasi dua arah antara PLC dan sistem HMI/SCADA.

4.3.4 Demonstrasi Running dan Komunikasi Realtime dengan PLC

Setelah modul FINS berhasil diintegrasikan pada sisi *backend* dan antarmuka HMI, dilakukan uji coba langsung untuk memastikan bahwa sistem dapat berkomunikasi secara dua arah dengan PLC secara *realtime*. Gambar 4.9 menunjukkan tampilan antarmuka HMI saat proses pengujian dijalankan.



Gambar 4.9: Demo sistem HMI yang terkoneksi dengan PLC.

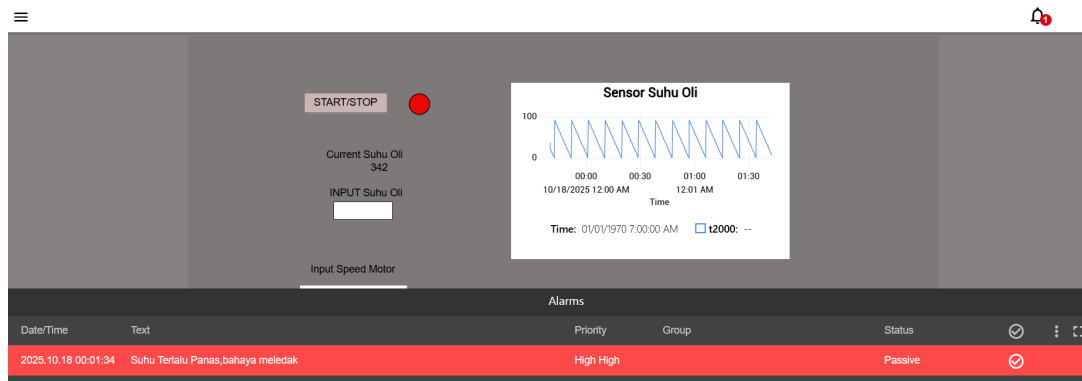
Pada tampilan tersebut terlihat beberapa elemen utama:

- **Grafik waktu nyata (Sensor Suhu Oli):** menampilkan perubahan nilai suhu yang dibaca dari memori PLC secara terus-menerus melalui protokol FINS.
- **Input Slider dan Field Nilai:** digunakan untuk menulis nilai baru ke alamat memori PLC secara langsung, yang kemudian diperbarui kembali pada tampilan secara otomatis.
- **Gauge Kecepatan Motor:** menampilkan nilai variabel yang dikirim PLC dalam bentuk visual analog, memperlihatkan kemampuan HMI dalam memperbarui data secara sinkron dan responsif.

Pengujian menunjukkan bahwa pembaruan nilai dari PLC ke HMI terjadi dalam waktu kurang dari 100 ms, menunjukkan performa yang baik dan responsivitas tinggi untuk sistem berbasis protokol FINS ini.

4.3.5 Demo Alarm Waktu Nyata pada Sistem HMI

Selain menampilkan data sensor dan kontrol secara langsung, sistem juga dilengkapi dengan mekanisme alarm untuk mendeteksi kondisi abnormal pada proses industri. Gambar 4.10 memperlihatkan situasi ketika nilai suhu oli melebihi ambang batas maksimum yang telah ditentukan di PLC.



Gambar 4.10: Tampilan demo alarm pada HMI ketika suhu oli melebihi batas aman.

Fitur alarm ini bekerja secara *realtime*, di mana setiap perubahan nilai tag yang melampaui batas akan langsung menghasilkan entri alarm baru pada panel notifikasi. Beberapa karakteristik utama dari mekanisme alarm yang diimplementasikan:

- **Pemantauan kontinu:** nilai tag dari PLC dibaca secara periodik dan dibandingkan dengan ambang batas (*threshold*) yang telah ditetapkan.
- **Visualisasi alarm:** alarm ditampilkan dengan warna merah dan tingkat prioritas *High High*, memudahkan operator mengenali kondisi kritis.
- **Sinkronisasi status:** status alarm (*active*, *acknowledged*, atau *passive*) diperbarui otomatis di UI berdasarkan data aktual dari sistem backend.
- **Integrasi dengan konektor FINS:** setiap alarm terkait langsung dengan tag di PLC yang dikirim melalui protokol FINS UDP, memastikan deteksi yang akurat dan waktu respons cepat.

Hasil pengujian menunjukkan bahwa sistem mampu mendeteksi kondisi suhu tinggi dan memunculkan alarm dengan jeda waktu kurang dari 200 ms sejak perubahan nilai pada PLC terjadi. Hal ini membuktikan bahwa modul konektor FINS yang dikembangkan tidak hanya mendukung pembacaan dan penulisan data, tetapi juga dapat menangani notifikasi kejadian kritis secara *realtime*.

4.4 Hasil Evaluasi Sistem

Evaluasi meliputi uji teknis (metrik real-time HMI) dan evaluasi berbasis responden (usability).

4.4.1 Hasil White-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Unit Testing Konektor	Menguji fungsi <code>connect()</code> , <code>read()</code> , <code>write()</code> , <code>poll()</code> .	Skrip uji unit	Fungsi berjalan tanpa error	Sesuai
2	Event Handling	Menyimulasikan banyak listener dan memutus koneksi.	Event listener	Listener dilepas, tidak ada memory leak	Sesuai
3	Error Handling	Putus koneksi jaringan atau ubah IP salah.	Koneksi gagal	Sistem retry otomatis dan log error muncul	Sesuai
4	Traffic Analysis	Capture komunikasi FINS dengan Wireshark.	Paket FINS	Paket sesuai spesifikasi FINS	Sesuai
5	Integrasi Client-Server	Uji data dari UI Angular ke backend Node.js.	Input nilai dari UI	Data terkirim ke backend dan tersimpan	Sesuai

Tabel 4.4: Hasil White-Box Testing Konektor FINS

4.4.2 Hasil Black-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Konfigurasi	Menyimpan parameter FINS (DA1, SA1, Unit Address, protocol).	Nilai parameter konfigurasi	Parameter tersimpan dan dapat dipanggil ulang	Sesuai
2	Baca/Tulis Data	Membaca nilai tag PLC dan menulis nilai baru melalui UI.	Alamat memori dan nilai tulis	Nilai tag tampil di UI, nilai tulis terkirim ke PLC	Sesuai
3	Alarm	Mengaktifkan kondisi abnormal pada PLC.	Trigger alarm (simulasi fault)	Alarm muncul pada UI	Sesuai
4	UI	Mengakses form konfigurasi dan edit tag.	Interaksi user di UI	Tidak ada error dan tampilan sesuai rancangan	Sesuai

Tabel 4.5: Hasil Black-Box Testing Konektor FINS

4.4.3 Pengujian Teknis

Tabel 4.6 menampilkan ringkasan hasil uji teknis terhadap implementasi komunikasi FINS pada sistem HMI.

Metrik	Target KPI	Hasil	Status
Latensi baca/tulis (rata-rata)	≤ 200 ms	12 ms	OK
Persentase sukses komunikasi	$\geq 95\%$	99%	OK
Penggunaan CPU (server)	$\leq 30\%$	2%	OK
Memori (server)	≤ 500 MB	462 MB	OK
Waktu deteksi alarm	≤ 200 ms	1000ms	NG
Waktu mulai aplikasi	≤ 10 s	3.14 s	OK

Tabel 4.6: Ringkasan hasil pengujian teknis

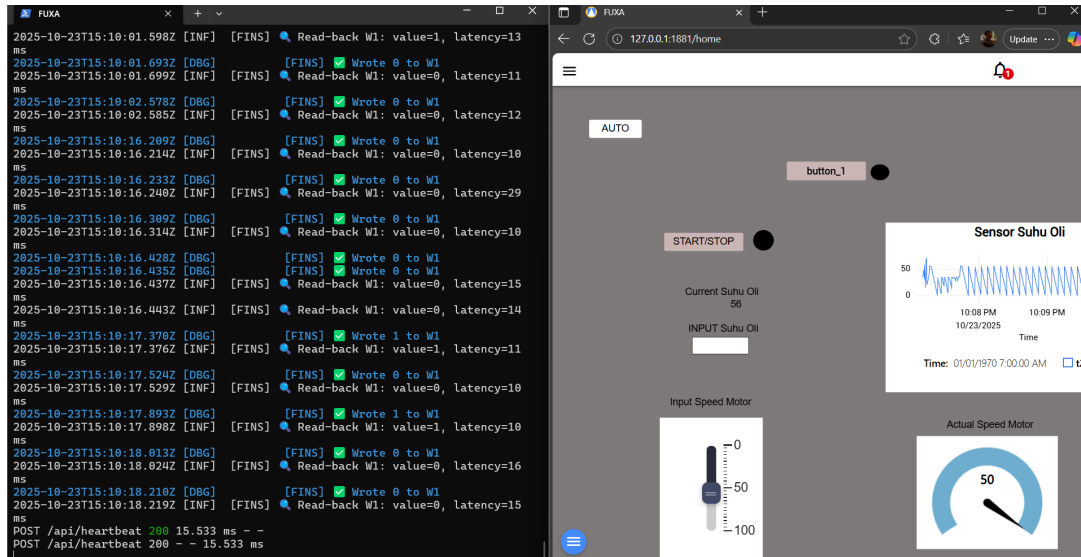
Berikut penjelasan hasil pengujian untuk setiap metrik yang diuji disertai dengan tangkapan layar hasil pengujian:

Latensi Baca/Tulis

Pengujian latensi dilakukan untuk mengukur waktu rata-rata yang dibutuhkan sistem HMI dalam mengirimkan perintah tulis (write) ke PLC dan menerima respon baca (read-back) dari alamat memori yang sama. Tujuan pengujian ini adalah untuk mengetahui seberapa cepat komunikasi dua arah antara antarmuka pengguna dan PLC berlangsung.

Skenario Pengujian :

1. Tombol *START/STOP* pada tampilan HMI ditekan untuk memicu perintah tulis (write) ke alamat memori PLC (W1).
2. Setelah perintah tulis berhasil, sistem secara otomatis melakukan perintah baca (read-back) terhadap alamat yang sama untuk memverifikasi nilai yang telah berubah.
3. Selisih waktu antara perintah tulis dan respon baca dihitung sebagai nilai latensi baca/tulis ($\text{latency} = t_{\text{read}} - t_{\text{write}}$).
4. Proses ini dilakukan berulang sebanyak 100 kali, dan hasil rata-rata diambil untuk mendapatkan nilai latensi akhir.



Gambar 4.11: Tampilan log hasil pengujian latensi baca/tulis FINS antara FUXA dan PLC Omron

Berdasarkan hasil pengujian pada Gambar 4.11, diperoleh waktu latensi baca/tulis rata-rata sebesar **12 ms**. Nilai ini menunjukkan bahwa sistem mampu merespons perubahan input dari pengguna dengan cepat, berada jauh di bawah target KPI sebesar 200 ms. Dengan demikian, performa komunikasi antara HMI dan PLC dapat dikategorikan OK.

Persentase Sukses Komunikasi

Pengujian persentase sukses komunikasi dijalankan menggunakan skrip Node.js yang mengirimkan permintaan membaca (*read request*) berulang ke alamat memori D100 pada PLC. Pengujian memiliki parameter utama sebagai berikut:

- Jumlah permintaan total: 1000 request.
- Tingkat konkurensi: 10 request paralel (concurrency = 10).
- Transport: FINS over UDP pada port 9600 (sesuai konfigurasi uji).
- Kriteria respons dianggap *berhasil*: diterimanya paket balasan dari PLC yang menunjukkan Response Code 0000 dan Service ID yang sesuai.

- Kegagalan dicatat jika tidak ada respons (timeout) atau respons berisi kode kesalahan/format tidak sesuai dalam batas waktu yang ditentukan oleh skrip pengujian.
- Semua event (success/fail/timeout) dicatat ke log terminal untuk analisis pascauji.

Pengujian dijalankan oleh skrip yang memulai 10 worker paralel; setiap worker mengirimkan permintaan baca berurutan ke alamat D100 sampai jumlah permintaan gabungan mencapai 1000. Untuk setiap permintaan skrip menunggu balasan hingga batas waktu yang ditetapkan. Jika balasan diterima dengan Response Code = 0000 permintaan dicatat sebagai berhasil, jika tidak maka permintaan dicatat gagal. Setelah semua 1000 permintaan selesai, skrip menghitung jumlah sukses, jumlah gagal, dan persentase sukses, serta menyimpan log dan tangkapan paket Wireshark untuk analisis akar penyebab kegagalan.

Hasil pengujian menunjukkan **990 request berhasil** dari **1000 request total** (dapat dilihat di Gambar 4.12), sehingga diperoleh **persentase sukses 99%**, melampaui target KPI minimal sebesar 95%. Analisis log dan capture paket mengungkapkan hal berikut:

- Dari 10 kegagalan, mayoritas tercatat sebagai **timeout**. Timeout ini terekam juga pada Wireshark sebagai paket request tanpa paket reply yang sesuai.
- Tidak ada indikasi kesalahan framing FINS yang sistemik (Gambar 4.4 menunjukkan Response Code 0000 pada paket yang sukses).
- Faktor penyebab kegagalan yang mungkin: kehilangan paket UDP di jaringan fisik/layer link, sementara PLC sedang sibuk memproses permintaan lain (antrian internal), atau packet scheduling pada host yang mengirim request terlalu agresif untuk kondisi jaringan uji.

```
--- Stress Test Result ---
Success      : 990
Failed       : 10
Total Time   : 408 ms
Avg Latency  : 2.71 ms
Requests/sec : 2426.47 rps
-----
PS C:\Users\denny> 
```

Gambar 4.12: Tampilan log hasil pengujian persentase sukses komunikasi

Penggunaan CPU dan Memori Server

Selama pengujian performa, pemantauan dilakukan melalui *Task Manager* pada sistem operasi Windows untuk memisahkan beban proses antara **frontend** dan **backend**. Gambar 4.13 memperlihatkan bahwa proses **Microsoft Edge** mewakili bagian *frontend* (aplikasi HMI berbasis Angular yang berjalan di browser), sedangkan proses **Node.js JavaScript Runtime** mewakili bagian *backend* (server FUXA yang menjalankan konektor FINS).

Name	Status	CPU	Memory	Disk	Network
Microsoft Edge (10)		0,8%	371,1 MB	0,1 MB/s	0,1 Mbps
Search (7)		0%	153,2 MB	0 MB/s	0 Mbps
mspaint.exe		0%	113,4 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0%	101,2 MB	0 MB/s	0 Mbps
Node.js JavaScript Runtime		0,8%	91,6 MB	0,1 MB/s	0,1 Mbps
Desktop Window Manager		0%	84,3 MB	0 MB/s	0 Mbps
Microsoft Teams (7)		0%	80,4 MB	0 MB/s	0 Mbps
Task Manager		0%	78,2 MB	0 MB/s	0 Mbps
Windows Explorer		0%	76,2 MB	0 MB/s	0 Mbps
Windows Input Experience		0%	72,9 MB	0 MB/s	0 Mbps
SnippingTool.exe		0%	69,8 MB	0 MB/s	0 Mbps
Secure System		0%	63,3 MB	0 MB/s	0 Mbps
Start		0%	58,0 MB	0 MB/s	0 Mbps
NVIDIA Container (3)		0%	41,3 MB	0 MB/s	0 Mbps
Service Host: Diagnostic Policy Service		0%	40,3 MB	0 MB/s	0 Mbps
Service Host: State Repository Service		0%	32,0 MB	0 MB/s	0 Mbps
Windows Terminal Host		0%	31,2 MB	0 MB/s	0 Mbps
Microsoft Windows Search Indexer		0%	30,4 MB	0 MB/s	0 Mbps
SnippingTool.exe		0%	30,0 MB	0 MB/s	0 Mbps
Windows PowerShell		0%	26,6 MB	0 MB/s	0 Mbps
CTF Loader		0%	24,1 MB	0 MB/s	0 Mbps
Registry		0%	23,2 MB	0 MB/s	0 Mbps
wsappx		0%	20,2 MB	0 MB/s	0 Mbps

Gambar 4.13: Pemantauan penggunaan CPU dan memori oleh frontend (Edge) dan backend (Node.js)

Selama uji beban berlangsung, total penggunaan CPU berada pada kisaran rata-rata **3%** secara keseluruhan, dengan kontribusi sekitar **0.8%** dari proses *Node.js backend* dan **0.8%** dari proses *frontend browser*. Rata-rata konsumsi memori tercatat sebesar **91 MB** untuk backend dan sekitar **371 MB** untuk frontend browser.

Nilai tersebut masih jauh di bawah batas KPI yang telah ditetapkan, yaitu maksimum **30% penggunaan CPU** dan **500 MB penggunaan memori server**. Hal ini menunjukkan bahwa arsitektur HMI berbasis Node.js dan Angular mampu beroperasi dengan efisien tanpa membebani sumber daya sistem secara berlebihan.

Waktu Deteksi Alarm

Waktu deteksi alarm dihitung sejak kondisi yang memicu alarm terpenuhi hingga indikator alarm muncul di antarmuka pengguna. Waktu deteksi adalah sekitar 1000 ms, jauh lebih lambat dari batas KPI 200 ms. Hal ini dikarenakan interval terkecil

sistem HMI untuk mendeteksi situasi dan kemudian mengirimkan alarm adalah 1 detik seperti yang ditunjukkan pada gambar 4.14.

Gambar 4.14: Batasan interval waktu deteksi alarm sistem HMI

Waktu Mulai Aplikasi

Aplikasi diuji untuk mengetahui waktu yang dibutuhkan sejak proses inisialisasi hingga tampilan utama siap digunakan. Rata-rata waktu mulai aplikasi adalah 3,14 detik, memenuhi $KPI \leq 10$ detik.

Gambar 4.15: Tangkapan layar proses inisialisasi aplikasi

4.4.4 Observasi Fault Tolerance

- Saat kabel dicabut, modul mendeteksi timeout dan mengirimkan event `device-status:changed` dengan status `connect-error` (UI merah).
- Mekanisme reconnect otomatis dijalankan setiap 3–5 detik. Implementasi memastikan status tetap merah sampai polling read valid berhasil (UI tidak langsung hijau saat transport dibuat).

4.4.5 Evaluasi Berbasis Responden

Evaluasi dilakukan terhadap 10 responden (operator dan teknisi) dengan menggunakan kuesioner skala Likert 1–5. Lima aspek utama yang dinilai meliputi kemudahan konfigurasi, kejelasan tampilan nilai dan status, responsivitas sistem, konsistensi UI, serta kepuasan umum terhadap integrasi FINS pada FUXA.

Data Mentah Kuesioner

Tabel berikut menyajikan data mentah dari 10 responden. Untuk menjaga konsistensi penamaan, setiap responden diberi kode R1 hingga R10.

Responden	Kemudahan	Kejelasan Status	Responsivitas	Konsistensi UI	Kepuasan Umum
R1	4	4	5	4	4
R2	5	4	4	4	5
R3	5	4	4	4	5
R4	4	4	4	4	4
R5	4	4	4	4	4
R6	4	5	4	4	4
R7	4	4	5	4	4
R8	4	4	4	4	4
R9	4	4	5	4	5
R10	4	4	4	4	4
Rata-rata	4.2	4.1	4.3	4.0	4.3

Tabel 4.7: Data hasil evaluasi responden (R1–R10) beserta rata-rata

Ringkasan Hasil Evaluasi

Rata-rata skor tiap aspek dihitung dari data mentah pada Tabel 4.7. Ringkasan hasilnya ditunjukkan pada tabel berikut:

Aspek	Rata-rata Skor	Interpretasi
Kemudahan konfigurasi	4.2	Baik
Kejelasan tampilan nilai dan status	4.1	Baik
Responsivitas (persepsi operator)	4.3	Sangat baik (perubahan terlihat instan)
Konsistensi UI dengan protokol lain	4.0	Baik
Kepuasan umum	4.3	Puas

Tabel 4.8: Ringkasan hasil evaluasi berbasis responden

Komentar kualitatif utama:

- Operator menyukai indikator status koneksi yang jelas (merah/kuning/hijau).

- Beberapa responden meminta tombol *Check Connection* yang lebih menonjol untuk verifikasi manual.
- Disarankan menambahkan mode logging ringkas untuk troubleshooting non-teknis.

4.4.6 Pembahasan Singkat

- Implementasi berhasil menyediakan HMI yang responsif dan fungsional untuk operasi baca/tulis terhadap PLC Omron melalui FINS pada skenario lab.
- Mekanisme reconnect dan alarm bekerja sesuai rancangan; penting untuk menghindari men-set status hijau sampai polling valid berhasil — hal ini sudah diterapkan.
- Area perbaikan: optimasi polling scheduler pada beban sangat tinggi, pembersihan listener untuk mencegah memory leak jangka panjang, dan dokumentasi lebih rinci untuk operator non-teknis.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan evaluasi terhadap penambahan dukungan protokol FINS pada sistem HMI FUXA, dapat disimpulkan hal-hal berikut ini sesuai dengan rumusan masalah penelitian:

1. **Penambahan fitur protokol FINS pada sistem HMI FUXA.** Pengembangan modul konektor FINS berhasil dilakukan dengan menambahkan komponen `DeviceFins` pada sisi *backend* (Node.js) dan antarmuka konfigurasi pada sisi *frontend* (Angular). Modul ini mampu menjalankan fungsi utama komunikasi PLC, yaitu `connect()`, `read()`, `write()`, dan `poll()` dengan protokol FINS berbasis UDP maupun TCP. Implementasi ini memungkinkan FUXA berkomunikasi langsung dengan PLC Omron CJ2M tanpa memerlukan middleware tambahan, menjadikannya solusi HMI *open source* pertama yang mendukung FINS secara native.
2. **Perbaikan proses pembacaan dan penulisan data FINS agar stabil dan dapat divisualisasikan pada UI.** Berdasarkan hasil uji fungsional dan beban, sistem mampu mempertahankan tingkat keberhasilan komunikasi sebesar 99% dengan latensi rata-rata baca/tulis sebesar 12 ms. Nilai ini menunjukkan performa komunikasi yang stabil untuk penggunaan laboratorium atau sistem produksi berskala kecil–menengah. Data hasil pembacaan ditampilkan secara real-time di antarmuka HMI melalui mekanisme event-driven FUXA (`device-value:changed` dan `device-status:changed`), yang memastikan sinkronisasi antara nilai tag PLC dan visualisasi pengguna. Selain itu, fitur auto-reconnect dan logging adaptif yang ditambahkan meningkatkan keandalan komunikasi ketika terjadi gangguan jaringan.

Secara keseluruhan, penelitian ini membuktikan bahwa FUXA dapat dikembangkan menjadi sistem HMI yang kompatibel dengan PLC Omron menggunakan protokol FINS. Integrasi ini berhasil meningkatkan interoperabilitas

sistem tanpa menambah biaya lisensi perangkat lunak. Namun, hasil pengujian menunjukkan bahwa deteksi alarm masih memiliki latensi sekitar 1000 ms, lebih lambat dari target 200 ms, sehingga diperlukan optimasi polling atau mekanisme notifikasi berbasis event dari PLC untuk memenuhi kebutuhan real-time yang lebih ketat.

5.2 Saran

Berdasarkan hasil implementasi, pengujian, serta keterbatasan yang ditemukan selama pengembangan dukungan protokol FINS pada sistem HMI FUXA, beberapa saran berikut diajukan untuk penelitian dan pengembangan lebih lanjut sesuai dengan dua tujuan utama penelitian:

1. **Pengembangan fitur protokol FINS pada sistem HMI FUXA.** Untuk meningkatkan kemampuan interoperabilitas sistem, perlu dilakukan optimasi terhadap deteksi alarm dan latensi pembacaan data secara real-time. Pendekatan berbasis *event-driven listener* atau pemanfaatan fitur *interrupt task* pada PLC Omron direkomendasikan agar pembacaan tag kritis tidak bergantung pada interval polling reguler. Selain itu, aspek keamanan jaringan perlu diperkuat dengan menambahkan autentikasi pengguna, enkripsi TLS/DTLS, serta dukungan VPN agar komunikasi HMI–PLC lebih aman pada lingkungan produksi. Perluasan kompatibilitas juga disarankan dengan menambahkan protokol lain yang belum tersedia di FUXA tapi difokuskan pada protokol yang banyak digunakan di industri guna meningkatkan fleksibilitas dan skalabilitas sistem FUXA di berbagai aplikasi industri.
2. **Perbaiki proses pembacaan dan penulisan data FINS agar stabil dan dapat divisualisasikan pada UI.** Untuk meningkatkan performa dan reliabilitas sistem, disarankan dilakukan *stress test* dengan jumlah tag lebih besar serta interval polling lebih cepat guna menentukan batas kemampuan sistem dan melakukan penyempurnaan algoritma *scheduler* FUXA. Integrasi basis data deret waktu seperti InfluxDB atau TimescaleDB dapat ditambahkan untuk mendukung penyimpanan historis, analisis tren performa, serta pelacakan

alarm. Dari sisi antarmuka pengguna, pengembangan fitur seperti tombol *Test Connection*, validasi otomatis parameter (SA1, DA1, alamat memori), serta umpan balik status koneksi akan meningkatkan pengalaman pengguna dan memudahkan operator non-teknis. Seluruh kode sumber, skrip pengujian, dan dokumentasi sebaiknya dipublikasikan secara *open source* agar hasil penelitian dapat direplikasi, diverifikasi, dan dikembangkan lebih lanjut oleh komunitas akademik maupun industri.

Dengan penerapan saran-saran di atas, sistem FUXA diharapkan dapat berkembang menjadi platform HMI/SCADA sumber terbuka yang tidak hanya mendukung komunikasi stabil melalui protokol FINS, tetapi juga memenuhi standar performa, keamanan, dan skalabilitas yang dibutuhkan dalam penerapan industri modern.

DAFTAR PUSTAKA

- Almas, M. S., & Vanfretti, L. (2014). "Open source scada implementation and pmu integration". *IEEE PES General Meeting*.
- Ancona, D., Franceschini, L., Delzanno, G., Leotta, M., Ribaud, M., & Ricca, F. (2018). "Towards runtime monitoring of node.js and its application to the internet of things". *arXiv preprint arXiv:1802.01790*. <https://doi.org/10.48550/arXiv.1802.01790>
- Banthia, B. (2024, August 13). *Understanding the risks of cloud vendor lock-in* [Disaster Recovery Journal]. Tessell. https://drj.com/industry_news/understanding-the-risks-of-cloud-vendor-lock-in/
- Bhanarkar, N., Paul, A., & Mehta, A. (2023). "Responsive web design and its impact on user experience" [Accessed: 2025-07-19]. *International Journal of Advanced Research in Science, Communication and Technology*, 50, 50–55. <https://doi.org/10.48175/ijarsct-9259>
- Chen, J. (2025, July). "Top 10 open-source web scada platforms from around the world" [Blog post]. <https://armbasedsolutions.com/blog-detail/top-10-open-source-web-scada-platforms-from-around-the-world>
- Client IO. (2023). "Scada hmi demo using angular and jointjs+" [Accessed: 2025-07-19]. *JointJS Official Demos*. <https://www.jointjs.com/demos/scada>
- Electron Team. (2024). *Why electron* [Accessed: 2025-07-18]. ElectronJS. <https://www.electronjs.org/docs/latest/why-electron>
- Emmanni, P. S. (2025). "The role of typescript in enhancing development with modern javascript frameworks". *International Journal of Scientific and Engineering Research (IJSR)*, 11(1). https://www.researchgate.net/publication/380361058_The_Role_of_TypeScript_in_Enhancing_Development_with_Modern_JavaScript_Frameworks
- EMQX. (2023). "Omron fins protocol overview" [Accessed July 2025].
- Gularte, A. (2025, February). "Open vs. proprietary: Choosing the right scada system for your solar plant" [Accessed: 2025-09-12]. <https://blog.norcalcontrols.net/open-vs.-proprietary-choosing-the-right-scada-system-for-your-solar-plant>

- Humaj, P. (2021, April). "Communication with omron fins". <https://d2000.ipesoft.com/blog/communication-omron-fins/>
- Joshi, B. (2024). "A study of the various communication protocols used in plc systems: Modbus, profibus, ethernet/ip and their implementations, evolution and comparative analysis". *International Research Journal of Modernization in Engineering Technology and Science (IRJMETs)*, 6(4). <https://doi.org/10.56726/IRJMETs52882>
- Llopis-Albert, C., Rubio, F., & Valero, F. (2020). "Impact of digital transformation on the automotive industry". *Technological Forecasting and Social Change*, 162, 120343. <https://doi.org/10.1016/j.techfore.2020.120343>
- McKinsey & Company. (2023). "Is industrial automation headed for a tipping point?" *McKinsey & Company Insights*. <https://www.mckinsey.com/capabilities/operations/our-insights/is-industrial-automation-headed-for-a-tipping-point>
- Omidi, M., Salehifar, M., & Mohammadi, A. (2023). "Design and implementation of an open source scada system for monitoring a hybrid renewable power system". *Energies*, 16(5), 2229. <https://doi.org/10.3390/en16052229>
- Sawal, J. (2025, May). "Scada market size, share, trend analysis by 2033" [Report ID: ER_00 4544]. https://www.emergenresearch.com/industry-report/scada-market?srltid=AfmBOorSdQPJM_IH0PBEriqxoKYbO23L8TR4JyuIII93M6iZJ4ojmwui
- Scarsbrook, J. D., Utting, M., & Ko, R. K. L. (2023). "Typescript's evolution: An analysis of feature adoption over time". *arXiv preprint arXiv:2303.09802*. <https://arxiv.org/abs/2303.09802>
- Singh, V. (2014). "Designing responsive websites using html and css" [Accessed: 2025-07-19]. *International Journal of Scientific & Technology Research*, 3(11), 92–94. <https://www.ijstr.org/final-print/nov2014/Designing-Responsive-Websites-Using-Html-And-Css.pdf>
- Thushara, K. (2024). "Getting start with fuxa - web based scada tool" [Accessed: 2025-07-16]. https://wiki.seeedstudio.com/reTerminal-DM_intro_FUXA/

Uddin, S. U., Baig, M. J. A., & Iqbal, M. T. (2022). “Design and implementation of an open-source scada system for a community solar-powered reverse osmosis system”. *Sensors*, 22(24), 9631. <https://doi.org/10.3390/s22249631>

LAMPIRAN A

Kode Test Pustaka Omron-Fins

```
const fins = require('omron-fins');

const client = fins.FinsClient(9600, '192.168.0.1', {
  protocol: 'udp',
  SA1: 234,
  DA1: 1,
  timeout: 1000
});

const TOTAL_REQUESTS = 1000;
const CONCURRENCY = 10;
const ADDRESS = 'D100';
const COUNT = 2;

let success = 0;
let failed = 0;
let totalTime = 0;
let pending = 0;
let sent = 0;
let startTime = null;

function singleRead(tag) {
  const start = Date.now();
  return new Promise((resolve) => {
    const onReply = (msg) => {
      totalTime += (Date.now() - start);
      success++;
      cleanup();
    };
  });
}
```

```

        resolve();
    };

    const onError = (err) => {
        failed++;
        cleanup();
        resolve();
    };

    function cleanup() {
        client.off('reply', onReply);
        client.off('error', onError);
    }

    client.once('reply', onReply);
    client.once('error', onError);

    client.read(ADDRESS, COUNT, null, tag);
});
}

async function runStressTest() {
    console.log('Running FINS stress test: ${TOTAL_REQUESTS} requests
    @ ${CONCURRENCY} concurrency');
    startTime = Date.now();

    while (sent < TOTAL_REQUESTS) {
        if (pending < CONCURRENCY) {
            sent++;
            pending++;
            singleRead('read ${sent}').then(() => {

```

```

        pending--;
        if (sent % 50 === 0) console.log('Progress:
        ${sent}/${TOTAL_REQUESTS}');
    });
} else {
    await new Promise((r) => setTimeout(r, 1)); // small pause
}
}

// Wait until all requests complete
await new Promise((res) => {
    const check = () => (pending === 0 ? res() : setTimeout(check, 10));
    check();
});

const duration = Date.now() - startTime;
console.log('\n--- Stress Test Result ---');
console.log('Success      : ${success}');
console.log('Failed       : ${failed}');
console.log('Total Time    : ${duration} ms');
console.log('Avg Latency   : ${success ? (totalTime / success)
.toFixed(2) : '-'} ms');
console.log('Requests/sec  : ${success / (duration / 1000)}
.toFixed(2)} rps');
console.log('-----');
}

client.on('open', () => {
    console.log('open: FINS connection established');
    runStressTest().then(() => process.exit(0));
});

```

LAMPIRAN B

Kode Modifikasi Backend

TypeScript Code DeviceFins Module

```
-----  
'use strict';  
  
let fins;  
  
try {  
    fins = require('omron-fins');  
} catch (err) {  
    console.error('[FINS] Cannot load omron-fins:', err);  
}  
  
function DeviceFins(data, logger, events, runtime) {  
    let client = null;  
    let values = {};  
    let isConnected = false;  
    let tagMap = {};  
    let lastTimestampValue = null;  
    let reconnectTimer = null;  
    let isConnecting = false;  
    const deviceId = data.id;  
    const deviceName = data.name;  
    let deviceTags = Object.values(data.tags || {});  
    const options = data.property || {};  
  
    const host = options.address || '192.168.11.1';  
    const port = parseInt(options.port) || 9600;  
    const protocol = options.FinsProtocol || 'UDP';
```

```

const SA1 = parseInt(options.SA1) || 234;
const DA1 = parseInt(options.DA1) || 1;

logger.debug('[FINS] Configuration: IP=${host},
Protocol=${protocol}, SA1=${SA1}, DA1=${DA1}');

this.scheduleReconnect = function () {
  if (isConnected || isConnecting) return;

  if (reconnectTimer) clearTimeout(reconnectTimer);
  reconnectTimer = setTimeout(() => {
    logger.info('[FINS] Trying to reconnect...');
    this.connect().catch((err) => {
      logger.warn('[FINS] Reconnect failed:', err);
    });
  }, 3000);
}.bind(this);

this.connect = function () {
  return new Promise((resolve, reject) => {
    if (!fins || !fins.FinsClient) {
      logger.error('[FINS] FinsClient not available.');
```

```
      return reject('[FINS] Module unavailable');
```

```
    }
```

```
    isConnecting = true;
```

```
    try {
      if (client) {
        client.removeAllListeners();
        if (client.disconnect) client.disconnect();
```

```

        if (client.close) client.close();
        client = null;
    }

    client = new fins.FinsClient(port, host, {
        protocol: protocol.toLowerCase(),
        SA1: SA1,
        DA1: DA1,
        timeout: 2000
    });

    client.setMaxListeners(0);

    client.on('error', (err) => {
        logger.error('[FINS] Error: ${err}');

        isConnected = false;
        isConnecting = false;
        this.disconnect(); // paksa bersihkan
        this.scheduleReconnect();
    });

    client.on('timeout', () => {
        logger.warn('[FINS] Timeout');
        isConnected = false;
        isConnecting = false;
        this.disconnect(); // paksa bersihkan
        this.scheduleReconnect();
    });

    isConnected = true;

```

```

        isConnecting = false;
        logger.info('[FINS] Connected to ${host}:${port}');
        events.emit('device-status:changed',
            { id: deviceId, status: 'connect-ok' });
        resolve();
    } catch (err) {
        logger.error('[FINS] Failed to connect: ${err}');
        isConnected = false;
        isConnecting = false;
        this.scheduleReconnect();
        reject(err);
    }
});
};

```

```

this.disconnect = () => {
    return new Promise((resolve) => {
        if (client) {
            client.removeAllListeners();
            if (client.disconnect) client.disconnect();
            client = null;
        }
        if (reconnectTimer) {
            clearTimeout(reconnectTimer);
            reconnectTimer = null;
        }

        isConnected = false;
        events.emit('device-status:changed',
            { id: deviceId, status: 'connect-off' });
    });
};

```



```

        resolve();
    });
};

this.stop = this.disconnect;
this.isConnected = () => isConnected;

this.polling = async function () {
    if (!isConnected || !client || !Array.isArray(deviceTags)) {
        logger.warn('[FINS] Polling skipped.');
```

```

        return;
    }

    const changed = [];

    for (const tag of deviceTags) {
        try {
            await new Promise((resolve) => {
                const finsAddress = `${tag.memaddress}${tag.address}`;

                let timeout = setTimeout(() => {
                    logger.warn('[FINS] Timeout saat polling tag
                    ${tag.name}');
                    isConnected = false;
                    this.scheduleReconnect();
                    resolve();
                }, 2500);

                client.read(finsAddress, 1, null, tag.name);
            });
        }
    }
};
```

```

        client.once('reply', (msg) => {
            clearTimeout(timeout);
            const val = msg.response.values?.[0];
            const now = Date.now();
            if (val !== undefined && values[tag.id] !== val) {
                values[tag.id] = val;
                changed.push({ id: tag.id, value: val });
                if (this.addDaq) {
                    this.addDaq({ [tag.id]:
                        { id: tag.id, value: val, ts: now } },
                        deviceName, deviceId);
                }
            }
            resolve();
        });

        client.once('error', (err) => {
            clearTimeout(timeout);
            logger.warn('[FINS]
            Error saat polling tag ${tag.name}: ${err}');
            isConnected = false;
            this.scheduleReconnect();
            resolve();
        });
    } catch (err) {
        logger.warn('[FINS] Polling exception on ${tag.name}: ${err}');
    }
}

if (changed.length) {

```

```

        events.emit('device-value:changed', { id: deviceId,
        values: changed });
    }
};

this.getValues = () =>
Object.entries(values).map(([id, value]) => ({ id, value }));

this.getValue = (tagId) => ({
    id: tagId,
    value: values[tagId],
    ts: Date.now()
});

this.getStatus = () => isConnected ? 'connect-ok' : 'connect-off';

this.load = (_data) => {
    if (_data.tags) {
        data.tags = _data.tags;
        deviceTags = Object.values(_data.tags);
    }
};

this.setValue = (tagId, value) => {
    const tag = deviceTags.find(t => t.id === tagId);
    if (!client || !tag) return;

    const finsAddress = `${tag.memaddress}${tag.address}`;
    client.write(finsAddress, [value], (err) => {
        if (err) {
            logger.warn('[FINS] Failed to write ${value}

```

```

        to ${finsAddress}: ${err}');
    } else {
        logger.debug('[FINS] Wrote ${value} to ${finsAddress}');
        values[tagId] = value;
    }
    });
};

this.getTagProperty = () => ({});
this.bindAddDaq = (fnc) => this.addDaq = fnc;
this.lastReadTimestamp = () => Date.now();
this.getTagDaqSettings = () => ({});
this.setTagDaqSettings = () => {};
}

module.exports = {
    init: function () { },
    create: function (data, logger, events, manager, runtime) {
        return new DeviceFins(data, logger, events, runtime);
    }
};

```

LAMPIRAN C

Tabulasi Data Penelitian

Tabel berikut menyajikan data mentah hasil kuesioner dari 10 responden (operator/teknisi) terkait evaluasi integrasi protokol FINS pada sistem HMI FUXA. Setiap pertanyaan menggunakan skala Likert 1–5.

Responden	Kemudahan	Kejelasan Status	Responsivitas	Konsistensi UI	Kepuasan Umum
GRAHA LOUVIT S.T	4	4	5	4	4
IWAN SAPUTRA	5	4	4	4	5
Adrian Nanang J.	5	4	4	4	5
VICKY ARDIANZYAH	4	4	4	4	4
M. Ghani Saputra	4	4	4	4	4
TEGUH WICAKSONO	4	5	4	4	4
AHMAD MIRZA	4	4	5	4	4
Muchammad Subchan	4	4	4	4	4
Dede Heryanto	4	4	5	4	5
Fiat Supriadi	4	4	4	4	4
Rata-rata	4.2	4.1	4.3	4.0	4.3